



Bayesian Sequential Skill Spaces for Sparse-Reward RL

Candidate Number: LTGZ8¹

MSc Machine Learning

Supervisors: Tim Rocktäschel & Yingchen Xu

Submission date: September 2025

¹**Disclaimer:** This report is submitted as part requirement for the MSc Machine Learning at University College London. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged.

Abstract

Sparse-reward, long-horizon environments such as NetHack/MiniHack challenge exploration and the preservation of reusable behaviours. This thesis proposes a Bayesian skill-space framework that couples a multi-modal variational autoencoder (VAE) with a sticky Hierarchical Dirichlet Process Hidden Markov Model (HDP-HMM) prior over the latent state, and integrates the resulting skill belief into a skill-aware PPO agent. A three-stage recipe — (i) unsupervised VAE pretraining, (ii) iterative VAE $\leftrightarrow$ HMM refinement, and (iii) online PPO with filtered skill beliefs — is paired with intrinsic rewards for dynamics-prediction surprise, gated skill-boundary entropy, and transition novelty. Diagnostics indicate the sticky HDP-HMM stabilises on-policy learning, induces persistent skill segments, and focuses curiosity on task-relevant novelty.

Keywords— multi-modal VAE - sticky HDP-HMM - Bayesian nonparametrics - hierarchical reinforcement learning - intrinsic motivation - MiniHack/NetHack - skill discovery - open-ended learning

Contents

1	Introduction	2
2	Background	6
2.1	Variational Autoencoders	6
2.1.1	Generative Latent Variable Model	6
2.1.2	Variational Inference	6
2.2	Bayesian Non-parametrics	7
2.2.1	Gaussian Process (GP)	8
2.2.2	Dirichlet Process (DP)	8
2.3	Hidden Markov Models	11
2.3.1	Generative story	12
2.3.2	Inference with fixed parameters	12
2.3.3	Parameter learning (Baum-Welch / EM)	13
2.4	Active Learning	14
2.4.1	Bayesian decision-theoretic formulation	14
2.4.2	Bayesian Active Learning by Disagreement (BALD)	14
2.5	Memorised Variational Bayes	15
2.5.1	Running example: DP mixture of exponentials	15
2.5.2	ELBO in terms of sufficient statistics	16
2.5.3	Parameter Updates	18
2.5.4	MemoVB: cache–update–replace loop	18
2.5.5	Complexity and advantages	18
2.6	Reinforcement Learning	19
2.6.1	Markov Decision Processes (MDPs)	19
2.6.2	Partially Observable MDPs (POMDPs)	20
2.6.3	Value-based Learning	21
2.6.4	Policy-based Learning	21
2.7	NetHack: a Roguelike With Structured, Symbolic State	24
2.7.1	Game overview	24
2.7.2	Observation tuple	24
2.7.3	Action space	26
2.7.4	Goal	26

3	Methodology	27
3.1	Multi-Modal Variational Autoencoder for MiniHack	28
3.1.1	Encoder and Decoder Architectures	28
3.1.2	Loss: Multi-term Reconstruction + β -Decomposed KL	30
3.1.3	β -Annealing and Stability	31
3.2	Skill Space via a Sticky Hierarchical DP HMM	32
3.3	On-Policy Skill-Aware PPO	34
3.3.1	Causal skill inference for control	35
3.3.2	Intrinsic Reward: Model-Uncertainty and Skill Entropy	36
3.3.3	PPO objective and recurrent update	38
3.4	Training Schedules	38
3.4.1	Stage I: VAE Pretraining	38
3.4.2	Stage II: Iterative Refinement of HMM-VAE	39
3.4.3	Stage III: Online PPO with VAE + HMM	40
4	Results and Analysis	42
4.1	VAE Pretraining	42
4.1.1	Effect of the prior on latent geometry	42
4.1.2	KL diagnostics under different priors	44
4.1.3	PCA/t-SNE comparison under different priors	46
4.2	Sticky HDP-HMM Fitting	48
4.3	PPO Ablations	49
4.3.1	MiniHack Room 15×15 : ablations and analysis	51
4.3.2	MiniHack River: transfer from Room and ablation analysis	54
5	Discussion	59
5.1	Broader task coverage and transfer	59
5.2	Stronger world models for NetHack	59
5.3	Improving the skill space	60
5.4	Skill-aware control	60
5.5	Stability for online learning	60
5.6	Evaluation and analysis at scale	61
5.7	Additional directions	61
6	Conclusion	62
A	NetHack Basic	64
A.1	blstats Description	64
A.2	Self-defined Semantic Class	65
B	Sticky HDP-HMM Variational Inference	66
B.1	Row-specific Transition Matrix Φ_k	66
B.2	HMM Forward-backward Message Passing	66
B.3	Emission Parameters η_k	67
B.4	Optimisation on Global Stick-breaking weights π	68

C	Training Hyperparameters	69
C.1	Stage I: VAE Pretraining	69
C.2	Stage II: Alternating HMM + VAE Refinement	70
C.3	Stage III: Online PPO with VAE + HMM	71
D	Declaration on LLM Use	73

Chapter 1

Introduction

Reinforcement learning (RL) has achieved remarkable success on short-horizon tasks with dense reward feedback [1], yet many real-world challenges involve long-horizon, sparse-reward sequential decision making. In such problems, agents must learn extended behaviours or skills to achieve distant goals using only infrequent scalar rewards as guidance [2, 3]. Classic examples include navigation or puzzle-solving games (e.g. Montezuma’s Revenge [4]) where the agent may receive a reward only after hundreds of steps. NetHack [5] and its variant MiniHack [6] exemplify this challenge: they are procedurally generated dungeon-crawling environments with high-dimensional observations, partial observability, and extremely sparse rewards for long-term objectives. In these settings, standard RL agents struggle to explore and discover multi-step strategies, often getting stuck dithering or repeating shallow patterns that yield no reward.

A promising approach to tame such complexity is hierarchical reinforcement learning (HRL) [7], which decomposes long tasks into reusable skills or options. By learning at multiple temporal scales, agents can plan with high-level skills, making credit assignment more tractable over long horizons. Prior works on the options framework formalized temporally extended actions with learned policies and termination conditions. Notably, the Option-Critic architecture [8] enabled learning of options end-to-end, and feudal networks [9] learned hierarchical policies by decomposing goals. Unsupervised skill discovery methods have also emerged: e.g. variational intrinsic control [10] and DIAYN [11] learn diverse skill policies by maximizing mutual information between latent skill variables and agent behaviour, without extrinsic rewards. These skills can later be reused or fine-tuned on downstream tasks, facilitating transfer learning and exploration. Another line of work, successor features, focuses on transferable representations of future outcomes [12]. Successor features decouple environment dynamics from reward functions, enabling an agent to rapidly transfer value functions across tasks with different rewards. While these approaches have shown benefits in skill discovery and knowledge transfer, applying them effectively to complex, open-ended environments like NetHack remains non-trivial. Challenges such as determining the right skill granularity, preventing the agent from forgetting earlier skills, and integrating learned skills into a goal-directed policy are still open problems.

In recent years, open-ended learning has been advocated as a route toward generally capable agents. Instead of training an agent for a single task, the idea is to expose it to an endless variety of tasks or an evolving curriculum so that it continually accumulates skills and knowledge. For

example, Open-Ended Learning Leads to Generally Capable Agents [13] demonstrated that by training agents in a procedurally generated 3D world with a universe of tasks, one can induce the emergence of versatile behaviours and heuristics. The MiniHack platform itself was proposed as a “sandbox for open-ended reinforcement learning research”, providing a suite of ever-more complex dungeon scenarios. These works underscore the importance of algorithms that can learn new skills in a self-driven way, remember and reuse those skills, and explore effectively even when external rewards are rare.

Sparse-reward environments often come with high-dimensional, multi-modal observations. In NetHack/MiniHack, the agent’s observation includes an ASCII grid of glyph characters and colors representing the dungeon, a host of game statistics (health, inventory, etc.), and textual messages – a complex multi-modal observation space. Learning a good state representation that captures the relevant features is essential. Poor representation learning can lead to agents “blind” to important aspects of the state, especially under partial observability. Recent work on world models and unsupervised representation learning in RL has shown the benefit of learned latent state spaces. Techniques such as Variational Autoencoders (VAE) [14] and Contrastive Predictive Coding (CPC) [15] allow agents to compress high-dimensional inputs into a low-dimensional latent code that retains predictive information about the environment. For example, Ha & Schmidhuber [16] trained a VAE and RNN as a world model to learn a latent representation of a racing game, enabling effective planning in the learned latent space. Similarly, in partially observable or procedurally generated games, having a compact state representation can improve generalization and stability of the learning process. In our setting, learning a multi-modal latent representation is particularly critical to subsequently identifying skills: if the latent state is too noisy or uninformative, any skill discovery method will struggle. Thus, a significant challenge is how to train a robust latent encoder for the complex observations of MiniHack in an unsupervised manner.

In an open-ended or sequential multi-task scenario, an agent might acquire some skills early on, but later face new subtasks that require learning new behaviours. A known danger in sequential learning is catastrophic forgetting — the agent’s neural network overwrites or loses the previously learned behaviours while optimizing for the new task. This is exacerbated if we fine-tune a single policy network on a sequence of tasks or if new skills interfere with old ones. Some works in continual learning and HRL have looked at preserving skills, for instance by freezing parts of the network [17] or by regularizing to preserve important weights [18]. In the context of skill discovery, one must ensure that once a useful skill is discovered, it is not unlearned when learning subsequent skills. Moreover, the agent should be able to recall and reuse the skill when relevant.

To tackle the above challenges, this thesis proposes a novel framework for open-ended skill discovery and exploitation in sparse-reward environments. The approach combines a multi-modal variational autoencoder (VAE) for representation learning with a sticky Hierarchical Dirichlet Process Hidden Markov Model (HDP-HMM) [19] for clustering the latent trajectories into discrete skills. This learned latent skill structure is then used to inform and regularize an actor-critic RL agent during policy learning, with the help of several intrinsic reward signals designed to encourage effective exploration. Figure 3.1 illustrates the overall architecture, which we summarize in three stages:

Stage I: Unsupervised Representation Learning with a Multi-Modal VAE. We first train a VAE to encode the rich observations of the MiniHack environment into a low-dimensional latent vector

z_t . Crucially, we do not use any task reward in this stage – the VAE is completely self-supervised, optimizing the evidence lower bound (ELBO) objective. Ensuring this latent is meaningful addresses the challenge of representation learning and provides a basis for measuring novelty or surprise.

Stage II: Latent Skill Discovery via a Sticky HDP-HMM. Instead of assuming a fixed prior on the latent states $z_{1:T}$ (as is usual in VAEs), we introduce a temporal prior in the form of a discrete HMM $h_{1:T}$ over an unknown number of “skill states”. The HDP-HMM is a Bayesian non-parametric extension of finite HMMs that can grow the number of states as needed, with a hierarchical Dirichlet Process prior tying together state transition probabilities. The “sticky” extension (parameter κ) biases the agent to remain in the same skill (self-transition) unless there is evidence to switch, which is appropriate for modelling skills that execute over multiple time steps. Importantly, this skill discovery is done in an unsupervised fashion, guided by the statistical structure of the latent state sequences rather than external rewards. The HDP aspect ensures we do not need to predefine the number of skills — the model can allocate new skill states if it encounters novel patterns, aligning with the open-ended learning objective.

Stage III: Skill-Aware Policy Learning with Intrinsic Rewards (Online RL). After establishing the VAE and HMM, we integrate them into the online reinforcement learning phase. We employ a Proximal Policy Optimization (PPO) algorithm [20] with an actor-critic architecture augmented to use the latent state and inferred skill as part of its input. To address the sparse extrinsic reward, we introduce several intrinsic reward signals that are added to the agent’s reward at each step (weighted appropriately). These intrinsic rewards are designed to encourage exploration and skill usage: (a) Dynamics-Prediction Surprise: using the VAE as a world model, we have the agent predict the next latent state from the current and reward it for prediction error. This is akin to the curiosity reward in the intrinsic curiosity module of Pathak et al. [21] or the prediction-based bonus in Random Network Distillation (RND) by Burda et al. [22] — it drives the agent to seek states that are surprising or hard to predict, thus exploring novel transitions. (b) Skill-Boundary Uncertainty: we compute the entropy of the HMM’s skill belief. When the agent is near a potential skill boundary, the entropy is higher. We use this as a bonus to encourage the agent to visit uncertain transitions, which often correspond to interesting situations. By rewarding high entropy at skill boundaries, we incentivise the agent to probe new skill transitions and refine its skill usage. (c) Skill Transition Novelty: we maintain a count or novelty measure for transitions between skills. Rare or new transitions yield a bonus, encouraging the agent to discover new sequences of skills. This is inspired by count-based exploration methods [23] extended to the space of skills: essentially, try doing skill A then skill B if you haven’t before. These intrinsic rewards, combined, form a curiosity-driven reward shaping that remains task-agnostic but helps the agent overcome sparse reward plateaus by self-motivating progress.

We evaluate our method on several MiniHack scenarios of increasing difficulty, demonstrating that our approach yields significant improvements in exploration and overall performance. In the MiniHack Room environment (a 15×15 empty maze where the agent must reach a goal stairway), our skill-discovery framework achieves about 8% higher success rate than a baseline PPO agent without the HMM and intrinsic rewards. In particular, the baseline agent (no HMM, no intrinsic bonuses) solves the task in roughly 37% of episodes, whereas our full approach reaches about 45% success. We also compare against a strong exploration baseline, Random Network Distillation

(RND), which rewards the agent for novel states. Our method outperforms the RND baseline on MiniHack Room (RND achieved 42% success in our experiments, whereas our full curiosity suite with HMM achieved 45.3%).

We further test the approach on the harder MiniHack River task, where the agent must push a boulder into water to form a bridge before reaching the hidden goal. This task involves at least two distinct high-level skills (“navigate around obstacles” and “push boulder”) in sequence, and the sparse reward is only given upon completing the entire sequence. Our approach, after being pre-trained on the Room task, achieves about 9% higher success on River compared to baselines. Specifically, with HMM and intrinsic rewards, the agent’s success rate reaches 43–47%, whereas the baseline was around 38%. In some runs, using just the HMM with a dynamics surprise bonus yielded the best performance (47% success), indicating the value of skill persistence for this multi-stage puzzle. Even the full curiosity suite with HMM (which includes the skill entropy and novelty bonuses) outperformed the no-HMM baseline by 4 points (42.7% vs 38.6%). These results demonstrate that incorporating a skill model and intrinsic motivation can substantially improve exploration and success in tasks that were previously unsolvable by standard RL in reasonable time.

The remainder of this thesis is organized as follows. Chapter 2 reviews the theoretical foundations and related work that underpin our approach. We cover variational autoencoders and variational inference, Bayesian non-parametric models, hidden Markov models, intrinsic motivation in reinforcement learning, and the NetHack learning environment. This establishes the context and tools for our method. Chapter 3 introduces our proposed framework in detail. We describe the design of the multi-modal VAE, the integration of the sticky HDP-HMM prior for skill discovery, the actor-critic architecture augmented with skill beliefs, and the intrinsic reward shaping strategy. The chapter is structured around the three training stages and includes algorithm pseudocode and model architecture diagrams. Chapter 4 presents our empirical results. We begin with a description of the experimental setup, including the MiniHack tasks, evaluation metrics, and baseline agents. We then report results on the MiniHack Room and River scenarios, with ablation studies that isolate the effect of the HMM and each intrinsic reward component. We also provide qualitative analyses, such as visualizing skill segments and agent trajectories. Chapter 5 provides a deeper analysis of our findings and discusses the limitations of our approach. We also situate our work in the broader context of open-ended learning, discussing how our approach could scale to larger problems or be combined with other techniques (such as option execution policies or model-based planning). Finally, Chapter 6 summarizes the contributions and key results of the thesis, and outlines directions for future work, including potential improvements to the skill discovery process, extensions to multi-agent settings, and applications to other complex games or real-world tasks.

Chapter 2

Background

This chapter is dedicated to providing statistical backgrounds for our work. We will have a brief introduction on variational autoencoders, Bayesian non-parametric models, hidden Markov models, active learning, memorised variational Bayes and actor-critic method for reinforcement learning.

2.1 Variational Autoencoders

2.1.1 Generative Latent Variable Model

Let $\mathbf{x} \in \mathbb{R}^D$ denote an observed datapoint and $\mathbf{z} \in \mathbb{R}^L$ an unobserved latent variable (usually $L < D$). A variational autoencoder (VAE) [14] defines the joint distribution

$$p_{\theta}(\mathbf{x}, \mathbf{z}) = p_{\theta}(\mathbf{x} \mid \mathbf{z})p(\mathbf{z}), \quad (2.1)$$

where $p(\mathbf{z})$ is a prior (e.g. $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$) on the latent space and the likelihood $p_{\theta}(\mathbf{x} \mid \mathbf{z})$ is parameterised by a neural network (the decoder) with weights θ . We denote observed data as $\mathcal{X} = \{\mathbf{x}_i\}$ and latent variables $\mathcal{Z} = \{\mathbf{z}_i\}$.

2.1.2 Variational Inference

In this model, the marginal likelihood $p_{\theta}(\mathcal{X})$ is completely intractable

$$\log p_{\theta}(\mathcal{X}) = \log \int p_{\theta}(\mathcal{X}, \mathcal{Z}) d\mathcal{Z},$$

but we can derive a lower bound of the marginal likelihood which we call Evidence Lower Bound (**ELBO**).

DEFINITION 1 (Evidence Lower Bound (ELBO)). *For any $q_{\phi}(\mathcal{Z}) \in \mathcal{Q}$,*

$$\mathcal{L}(\phi, \theta; \mathcal{X}) := \mathbb{E}_{q_{\phi}(\mathcal{Z})} [\log p_{\theta}(\mathcal{X}, \mathcal{Z})] + \mathbf{H}_q \quad (2.2)$$

where $\mathbf{H}_q$ is the entropy of q ,

$$\mathbf{H}_q := - \int \log q_\phi(\mathcal{Z}) q_\phi(\mathcal{Z}) d\mathcal{Z},$$

$\mathcal{L}(\phi, \theta; \mathcal{X})$ is a lower bound on $\log p_\theta(\mathcal{X})$, called the evidence lower bound (ELBO).

Proof. We have log likelihood

$$\begin{aligned} \ell(\theta) &= \log p_\theta(\mathcal{X}) = \log \int p_\theta(\mathcal{X}, \mathcal{Z}) d\mathcal{Z} \\ &= \log \int q_\phi(\mathcal{Z}) \frac{p_\theta(\mathcal{X}, \mathcal{Z})}{q_\phi(\mathcal{Z})} d\mathcal{Z} \geq \int q_\phi(\mathcal{Z}) \log \frac{p_\theta(\mathcal{X}, \mathcal{Z})}{q_\phi(\mathcal{Z})} d\mathcal{Z} =: \mathcal{L}(\phi, \theta; \mathcal{X}). \end{aligned}$$

□

Instead of maximising intractable log likelihood, one can maximise ELBO with approximated posterior inference. VAEs introduce an encoder network $q_\phi(\mathcal{Z} \mid \mathcal{X})$ with parameters ϕ to approximate it.

ELBO can be rewritten as

$$\begin{aligned} \mathcal{L}(\phi, \theta; \mathcal{X}) &= \log p_\theta(\mathcal{X}) - D_{\text{KL}} [q_\phi(\mathcal{Z}) \parallel p_\theta(\mathcal{Z} \mid \mathcal{X})] \\ &= \underbrace{\mathbb{E}_{q_\phi(\mathcal{Z})} [\log p_\theta(\mathcal{X} \mid \mathcal{Z})]}_{\text{reconstruction cost}} - \underbrace{D_{\text{KL}} [q_\phi(\mathcal{Z}) \parallel p(\mathcal{Z})]}_{\text{regulariser}} \end{aligned}$$

where D_{KL} is Kullback–Leibler divergence

$$D_{\text{KL}} [Q \parallel P] = \int Q \log \frac{Q}{P} \geq 0,$$

with $D_{\text{KL}} [Q \parallel P] = 0 \iff Q = P$ as measures. Maximising ELBO is equivalent to minimising KL-divergence of variational posterior with true posterior. It can be also written as reconstruction cost with regulariser. The reconstruction cost measures how well the encoding-decoding process and the regulariser penalises large movement away from prior distribution. Given an observation $\mathbf{x}$, the variational family is the set usually chosen as a multivariate Gaussian with diagonal covariance $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}[\boldsymbol{\sigma}_\phi^2(\mathbf{x})])$.

2.2 Bayesian Non-parametrics

Bayesian non-parametrics (BNP) place priors on infinite-dimensional objects—e.g. functions or random measures—so model capacity can grow with data. Typical motivations are

- flexibility (no a-priori limit on complexity),
- principled uncertainty quantification, and
- automatic control of model size.

Below we sketch two canonical BNP priors.

2.2.1 Gaussian Process (GP)

DEFINITION 2 (Gaussian Process). *A Gaussian Process (GP) is a collection of random variables $\{f(\mathbf{x})\}_{\mathbf{x} \in \mathcal{X}}$ such that any finite restriction $(f(\mathbf{x}_1), \dots, f(\mathbf{x}_n))$ is multivariate normal. A GP is fully specified by a mean $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$ and covariance (kernel) function $k(\mathbf{x}, \mathbf{x}') = \text{cov}(f(\mathbf{x}), f(\mathbf{x}'))$, denoted*

$$f \sim \mathcal{GP}(m(\cdot), k(\cdot, \cdot)).$$

Given observations $y_i \mid \mathbf{x}_i, f \sim \mathcal{N}(f(\mathbf{x}_i), \sigma^2)$ and let $\mathbf{K}_{XX}$ be the Gram matrix $[k(\mathbf{x}_i, \mathbf{x}_j)]_{ij}$, then

$$\mathbf{y} \sim \mathcal{N}(m(X), \tilde{\mathbf{K}}_{XX})$$

with $\tilde{\mathbf{K}}_{XX} = \mathbf{K}_{XX} + \sigma^2 I$.

Posterior From [24], the posterior latent f is also a GP,

$$f(\cdot) \mid \mathbf{y}, \mathbf{X} \sim \mathcal{N}\left(m(\cdot) + \mathbf{K}_{\cdot X} \tilde{\mathbf{K}}_{XX}^{-1}(\mathbf{y} - \mathbf{m}), k(\cdot, \cdot) - \mathbf{K}_{\cdot X} \tilde{\mathbf{K}}_{XX}^{-1} \mathbf{K}_{X \cdot}\right) \quad (2.3)$$

Prediction Conditional on observed points $\mathbf{y}$ and X , we can predict the new point $\mathbf{x}$ as normal distribution as well.

$$y \mid \mathbf{y}, X, \mathbf{x} \sim \mathcal{N}\left(m(\mathbf{x}) + \mathbf{K}_{\mathbf{x} X} \tilde{\mathbf{K}}_{XX}^{-1}(\mathbf{y} - \mathbf{m}), k(\mathbf{x}, \mathbf{x}) - \mathbf{K}_{\mathbf{x} X} \tilde{\mathbf{K}}_{XX}^{-1} \mathbf{K}_{X \mathbf{x}} + \sigma^2\right) \quad (2.4)$$

Popular kernels include squared-exponential, Matérn, and periodic.

2.2.2 Dirichlet Process (DP)

DEFINITION 3 (Dirichlet Process [25]). *A random probability measure G on $(\Theta, \mathcal{B})$ follows a Dirichlet Process with concentration $\alpha > 0$ and base measure G_0 , written $G \sim \text{DP}(\alpha, G_0)$, if for every finite measurable partition $(A_1, \dots, A_K)$*

$$(G(A_1), \dots, G(A_K)) \sim \text{Dir}(\alpha G_0(A_1), \dots, \alpha G_0(A_K)). \quad (2.5)$$

The base measure G_0 is basically the mean of the DP: for any measurable set $A \subset \Theta$, we have

$$\mathbb{E}[G(A)] = G_0(A) \quad \text{Var}(G(A)) = \frac{G_0(A)(1 - G_0(A))}{\alpha + 1},$$

the larger α is, the smaller the variance and DP will concentrate more of its mass around the mean. The concentration parameter is also called the strength parameter [26].

Posterior Let $G \sim \text{DP}(\alpha, G_0)$, $\theta_1, \dots, \theta_n$ be a sequence of independent draws from G , and let $n_k = \#\{i : \theta_i \in A_k\} = \sum_{i=1}^n \delta_{\theta_i}(A_k)$ be the number of observed values in partition A_k , where δ_{θ_i} is dirac-delta function located at θ_i . By (2.5) and the conjugacy between the Dirichlet and the multinomial distributions, we have:

$$(G(A_1), \dots, G(A_K)) \mid \theta_1, \dots, \theta_n \sim \text{Dir}(\alpha G_0(A_1) + n_1, \dots, \alpha G_0(A_K) + n_K). \quad (2.6)$$

So for posterior DP, we have

$$G \mid \theta_1, \dots, \theta_n \sim \text{DP} \left(\alpha + n, \frac{\alpha}{\alpha + n} G_0 + \frac{n}{\alpha + n} \frac{\sum_{i=1}^n \delta_{\theta_i}}{n} \right) \quad (2.7)$$

Prediction Given we have i.i.d. sequence $\theta_1, \dots, \theta_n \sim G$, consider the predictive distribution for θ_{n+1} conditioned on the observed data, we have

$$\begin{aligned} P(\theta_{n+1} \in A \mid \theta_1, \dots, \theta_n) &= \mathbb{E}[G(A) \mid \theta_1, \dots, \theta_n] \\ &= \frac{1}{\alpha + n} \left(\alpha G_0(A) + \sum_{i=1}^n \delta_{\theta_i}(A) \right) \end{aligned}$$

So the prediction is a categorical distribution

$$\theta_{n+1} \mid \theta_1, \dots, \theta_n \sim \frac{1}{\alpha + n} \left(\alpha G_0 + \sum_{i=1}^n \delta_{\theta_i} \right) \equiv \frac{\alpha}{\alpha + n} G_0 + \frac{n}{\alpha + n} \frac{\sum_{i=1}^n \delta_{\theta_i}}{n} \quad (2.8)$$

So a new point is drawn with probability $\frac{\alpha}{\alpha+n}$ from base measure G_0 or with probability $\frac{n}{\alpha+n}$ from existing point mass. This is called Blackwell-MacQueen Urn Scheme [27]. We can construct joint distribution over sequences $\theta_1, \dots, \theta_n$ by iteratively drawing each θ_i given $\theta_1, \dots, \theta_{i-1}$, specifically

$$p(\theta_1, \dots, \theta_n) = \prod_{i=1}^n p(\theta_i \mid \theta_1, \dots, \theta_{i-1}). \quad (2.9)$$

Infinite Exchangeability From (2.8) we can observe the predictive distribution depends only on the multiset of previous values (the counts) not on their arrival order, so any permutation of the order of appearance gives exactly the same joint distribution. Given any permutation σ on $1, \dots, n$, we have

$$p(\theta_1, \dots, \theta_n) = p(\theta_{\sigma(1)}, \dots, \theta_{\sigma(n)}). \quad (2.10)$$

We can also check DP sequence has consistent distributions.

DEFINITION 4. *If for every $n \in \mathbb{N}$, a probability distribution μ_n on the first n coordinates*

$$(\theta_1, \dots, \theta_n) \in \Theta^n$$

such that

$$\mu_{n+1}(A \times \Theta) = \mu_n(A) \quad \forall A \subseteq \Theta^n,$$

then the family $\{\mu_n\}$ is consistent.

Specifically for DP sequence,

$$\int_{\Theta} p(\theta_1, \dots, \theta_n, \theta_{n+1}) d\theta_{n+1} = \int_{\Theta} p(\theta_1, \dots, \theta_n) p(\theta_{n+1} \mid \theta_1, \dots, \theta_n) d\theta_{n+1} = p(\theta_1, \dots, \theta_n).$$

By Kolmogorov extension theorem, there exists a unique probability measure μ on the infinite product space Θ^∞ whose first n marginals are exactly μ_n . Together with (2.10), the theorem

guarantees an infinite-sequence measure that is invariant under any finite permutation. Consequently, the sequence is infinitely exchangeable.

Discreteness Property (2.8) implies the distribution G has point masses. For a long enough sequence of draws from G , $\frac{n}{\alpha+n}$ dominates, and hence the value of any draw will be repeated by another law, which implies G only consists of a weighted sum of point masses. G is a discrete distribution.

Clustering Property Let $\theta_1^*, \dots, \theta_m^*$ be the unique values among $\theta_1, \dots, \theta_n$ and n_k be the number of repeats of θ_k^* . (2.8) can be rewritten as

$$\theta_{n+1} \mid \theta_1, \dots, \theta_n \sim \frac{1}{\alpha + n} \left(\alpha G_0 + \sum_{k=1}^m n_k \delta_{\theta_k^*} \right). \quad (2.11)$$

Note the larger n_k is, the higher probability that θ_k^* will grow. This is a rich-gets-richer phenomenon. Also note from (2.8) the observation θ_i takes on a new value with probability $\frac{\alpha}{\alpha+i-1}$. Write $Z_i = \mathbf{1}\{\theta_i \text{ starts a new cluster}\}$ and M_n as the number of clusters after n observations. $M_n = \sum_{i=1}^n Z_i$. We have

$$P(Z_i = 1 \mid \theta_{1:i-1}) = \frac{\alpha}{\alpha + i - 1} =: p_i, \quad i \geq 2,$$

M_n is sum of independent Bernoulli distribution. So we have

$$\mathbb{E}[M_n] = \sum_{i=1}^n p_i = \alpha \sum_{i=0}^{n-1} \frac{1}{\alpha + i} = \alpha[\psi(\alpha + n) - \psi(\alpha)],$$

where $\psi(x)$ is digamma function defined as $\psi(x) = \frac{d}{dx} \log \Gamma(x)$ and using the digamma identity

$$\psi(x + N) - \psi(x) = \sum_{k=0}^{N-1} \frac{1}{x + k}.$$

For variance we have

$$\begin{aligned} \text{Var}(M_n) &= \sum_{i=1}^n p_i(1 - p_i) = \sum_{i=1}^n p_i - \sum_{i=1}^n p_i^2, \\ \sum_{i=1}^n p_i^2 &= \alpha^2 \sum_{i=0}^{n-1} \frac{1}{(\alpha + i)^2} = \alpha^2[\psi'(\alpha) - \psi'(\alpha + n)], \end{aligned}$$

where $\psi'(x) = \frac{d}{dx} \psi(x)$ is trigamma function and it satisfies $\psi'(x) = \sum_{k=0}^{\infty} \frac{1}{(x+k)^2}$. Putting together we have

$$\text{Var}(M_n) = \alpha[\psi(\alpha + n) - \psi(\alpha)] + \alpha^2[\psi'(\alpha + n) - \psi'(\alpha)].$$

For sufficiently large n , we have asymptotic results

$$\mathbb{E}[M_n] \simeq \alpha \log \left(1 + \frac{n}{\alpha} \right), \quad \text{Var}(M_n) \simeq \alpha \log \left(1 + \frac{n}{\alpha} \right),$$

so the cluster grows logarithmically in the number of observation.

Stick-breaking Construction Since a DP is composed of a weighted sum of point masses. [28] provides a constructive definition of DP called the stick-breaking construction. It is given as follows

$$\begin{aligned}\beta_k &\sim \text{Beta}(1, \alpha), \\ \theta_k^* &| G_0 \sim G_0, \\ \pi_k &= \beta_k \prod_{l=1}^{k-1} (1 - \beta_l), \\ G &= \sum_{k=1}^{\infty} \pi_k \delta_{\theta_k^*}.\end{aligned}\tag{2.12}$$

Then $G \sim \text{DP}(\alpha, G_0)$. The stick-breaking distribution over π is sometimes written as

$$\pi \sim \text{GEM}(\alpha),$$

where GEM is Griffiths, Engen and McCloskey Distribution [29].

Dirichlet-Process Mixture Model (DPMM) Dirichlet-process mixture model is the most common application of the Dirichlet process. The non-parametric nature of Dirichlet process translates to mixture models with a countably infinite number of components. Given a set of observations $\{x_1, \dots, x_n\}$ with latent parameters $\{\theta_1, \dots, \theta_n\}$. Each θ_i is i.i.d. from G and each x_i has distribution $F(\theta_i)$ parametrised by θ_i .

$$\begin{aligned}x_i &| \theta_i \sim F(\theta_i), \\ \theta_i &| G \sim G, \\ G &| \alpha, G_0 \sim \text{DP}(\alpha, G_0).\end{aligned}\tag{2.13}$$

This gives a countably-infinite mixture with an automatically inferred number of components. (2.13) can be expressed equivalently with stick-breaking construction.

$$\begin{aligned}\pi &| \alpha \sim \text{GEM}(\alpha), \\ s_i &| \pi \sim \text{Cat}(\pi), \\ \theta_k^* &| G_0 \sim G_0, \\ x_i &| s_i, \{\theta_k^*\} \sim F(\theta_{s_i}^*),\end{aligned}\tag{2.14}$$

with $G = \sum_{k=1}^{\infty} \pi_k \delta_{\theta_k^*}$ and $\theta_i = \theta_{s_i}^*$.

2.3 Hidden Markov Models

A hidden Markov model is a doubly-stochastic generative model for a sequence of latent states $S_{1:T} = \{s_1, \dots, s_T\}$ and observations $\mathbf{X}_{1:T} = \{\mathbf{x}_1, \dots, \mathbf{x}_T\}$:

- The states form a first-order Markov chain $p(s_t | s_{t-1}, \dots, s_1) = p(s_t | s_{t-1})$.
- Each observation $\mathbf{x}_t$ depends only on the current state s_t .

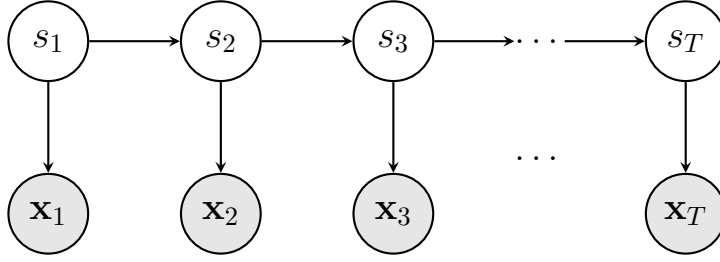


Figure 2.1: Graphical model of a Hidden Markov Model with explicit ellipsis indicating repetition.

2.3.1 Generative story

Let the parameter set be $\lambda = (\boldsymbol{\pi}, \boldsymbol{\Phi}, \mathbf{A})$ where

- $\pi_i = P(s_1 = i)$, initial-state distribution,
- $\Phi_{ij} = P(s_t = j \mid s_{t-1} = i)$, state-transition matrix,
- $A_j(\mathbf{x}_t) = P(\mathbf{x}_t \mid s_t = j)$, emission density for continuous $\mathbf{x}_t$, or
- $A_{jk} = P(\mathbf{x}_t = k \mid s_t = j)$ for discrete $\mathbf{x}_t$.

The joint density factorises as

$$p_{\lambda}(s_{1:T}, \mathbf{x}_{1:T}) = p(s_1)p(\mathbf{x}_1 \mid s_1) \prod_{t=2}^T p(s_t \mid s_{t-1})p(\mathbf{x}_t \mid s_t) \quad (2.15)$$

Typical emission choices are categorical distributions for discrete data and Gaussians (or GMMs) for continuous data.

2.3.2 Inference with fixed parameters

Forward recursion In HMM, we have

$$p(s_t \mid \mathbf{x}_{1:t}) \propto p(\mathbf{x}_t \mid s_t) \int p(s_t \mid s_{t-1})p(s_{t-1} \mid \mathbf{x}_{1:t-1})ds_{t-1}.$$

We can define $\alpha_t(i) := p(\mathbf{x}_{1:t}, s_t = i)$ and we have

$$\begin{aligned} \alpha_1(i) &= \pi_i A_i(\mathbf{x}_1), \\ \alpha_{t+1}(i) &= A_i(\mathbf{x}_{t+1}) \sum_{j=1}^K \Phi_{ji} \alpha_t(j), \quad t \geq 1. \end{aligned} \quad (2.16)$$

We have posterior and joint density

$$p(s_t = i \mid \mathbf{x}_{1:t}) = \frac{\alpha_t(i)}{\sum_k \alpha_t(k)}, \quad (2.17)$$

$$p(\mathbf{x}_{1:T}) = \sum_{s_T} p(\mathbf{x}_{1:T}, s_T) = \sum_{k=1}^K \alpha_T(k). \quad (2.18)$$

Backward recursion We define $\beta_t(i) := p(\mathbf{x}_{t+1:T} \mid s_t = i)$ and we have

$$\begin{aligned}
\beta_T(i) &= 1, \\
\beta_t(i) &= p(\mathbf{x}_{t+1:T} \mid s_t = i) \\
&= \sum_{j=1}^K p(s_{t+1} = j, \mathbf{x}_{t+1}, \mathbf{x}_{t+2:T} \mid s_t = i) \\
&= \sum_{j=1}^K p(s_{t+1} = j \mid s_t = i) p(\mathbf{x}_{t+1} \mid s_{t+1} = j) p(\mathbf{x}_{t+2:T} \mid s_{t+1} = j) \\
&= \sum_{j=1}^K \Phi_{ij} A_j(\mathbf{x}_{t+1}) \beta_{t+1}(j).
\end{aligned} \tag{2.19}$$

Posterior marginals We define $\gamma_t(i) := p(s_t = i \mid \mathbf{x}_{1:T})$ and we have

$$\gamma_t(i) = P(s_t = i \mid \mathbf{x}_{1:T}) = \frac{\alpha_t(i) \beta_t(i)}{\sum_k \alpha_t(k) \beta_t(k)}. \tag{2.20}$$

Viterbi Algorithm If we would like to find single best path, i.e.

$$\arg \max_{s_{1:T}} p(s_{1:T} \mid \mathbf{x}_{1:T}, \lambda).$$

We simply replace the summations in (2.16) by maxima to obtain the Viterbi dynamic programme,

$$\alpha_t^{\max}(i) = A_i(\mathbf{x}_t) \max_j \alpha_{t-1}^{\max}(j) \Phi_{ji}.$$

2.3.3 Parameter learning (Baum-Welch / EM)

We define pairwise posterior

$$\begin{aligned}
\xi_t(i, j) &:= p(s_t = i, s_{t+1} = j \mid \mathbf{x}_{1:T}) = \frac{p(\mathbf{x}_{t+1:T} \mid s_{t+1} = j) p(s_{t+1} = j \mid s_t = i) p(s_t = i \mid \mathbf{x}_{1:t})}{p(\mathbf{x}_{t+1:T} \mid \mathbf{x}_{1:t})} \\
&\propto \beta_{t+1}(j) A_j(\mathbf{x}_{t+1}) \Phi_{ij} \alpha_t(i), \\
\sum_j \xi_t(i, j) &= \gamma_t(i).
\end{aligned} \tag{2.21}$$

In M-step, we update

$$\begin{aligned}
\pi_i^{\text{new}} &= \gamma_1(i), \\
\Phi_{ij}^{\text{new}} &= \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}, \\
A_i^{\text{new}} &= \arg \max_A \sum_{t=1}^T \gamma_t(i) \log A(\mathbf{x}_t).
\end{aligned} \tag{2.22}$$

where the last step has a closed form for exponential-family emissions. Specifically for the

Gaussian emission,

$$\begin{aligned}\boldsymbol{\mu}_i^{\text{new}} &= \frac{\sum_{t=1}^T \gamma_t(i) \mathbf{x}_t}{\sum_{t=1}^T \gamma_t(i)}, \\ \Sigma_i^{\text{new}} &= \frac{\sum_{t=1}^T \gamma_t(i) (\mathbf{x}_t - \boldsymbol{\mu}_i^{\text{new}})(\mathbf{x}_t - \boldsymbol{\mu}_i^{\text{new}})^\top}{\sum_{t=1}^T \gamma_t(i)}.\end{aligned}\tag{2.23}$$

2.4 Active Learning

Let $\mathcal{U} = \{\mathbf{x}^{(i)}\}_{i=1}^M$ be an unlabelled pool and $\mathcal{D}_n = \{(\mathbf{x}_j, y_j)\}_{j=1}^n$ the labelled set at round n . An acquisition rule $\alpha(\mathbf{x}; \mathcal{D}_n)$ scores each $\mathbf{x} \in \mathcal{U}$; we query the oracle for the label of $\mathbf{x}^* = \arg \max_{\mathbf{x}} \alpha(\mathbf{x}; \mathcal{D}_n)$, append $(\mathbf{x}^*, y^*)$ to $\mathcal{D}_n$, and retrain.

2.4.1 Bayesian decision-theoretic formulation

With model parameters $\boldsymbol{\theta}$ and loss $L(\boldsymbol{\theta}; \mathcal{D})$, an optimal Bayesian query maximises the expected reduction in posterior risk

$$\alpha^{\text{Bayes}}(\mathbf{x}) = \mathbb{E}_{y|\mathbf{x}, \mathcal{D}_n} \left[L(\boldsymbol{\theta}; \mathcal{D}_n) - L(\boldsymbol{\theta}; \mathcal{D}_n \cup \{(\mathbf{x}, y)\}) \right].$$

Exact evaluation is usually intractable; BALD provides a tractable information-theoretic surrogate.

2.4.2 Bayesian Active Learning by Disagreement (BALD)

BALD selects the example whose label would yield the largest mutual information between predictions and model parameters [30]:

DEFINITION 5 (BALD acquisition score).

$$\alpha_{\text{BALD}}(\mathbf{x}; \mathcal{D}_n) = \mathbf{I}[y, \boldsymbol{\theta} \mid \mathbf{x}, \mathcal{D}_n] = \mathbf{H}[y \mid \mathbf{x}, \mathcal{D}_n] - \mathbb{E}_{\boldsymbol{\theta}|\mathcal{D}_n} \left[\mathbf{H}[y \mid \mathbf{x}, \boldsymbol{\theta}] \right], \tag{2.24}$$

where $\mathbf{I}$ denotes mutual information and is defined as

$$\mathbf{I}[X, Y] = D_{\text{KL}}[p(x, y) \| p(x)p(y)],$$

$\mathbf{H}$ denotes Shannon entropy and the expectation is taken over the posterior $p(\boldsymbol{\theta} \mid \mathcal{D}_n)$.

To derive (2.24), for a candidate $\mathbf{x}$,

$$\begin{aligned}
\mathbf{I}[y, \boldsymbol{\theta} \mid \mathbf{x}, \mathcal{D}_n] &= \int p(y, \boldsymbol{\theta} \mid \mathbf{x}, \mathcal{D}_n) \log \frac{p(y, \boldsymbol{\theta} \mid \mathbf{x}, \mathcal{D}_n)}{p(y \mid \mathbf{x}, \mathcal{D}_n) p(\boldsymbol{\theta} \mid \mathcal{D}_n)} dy d\boldsymbol{\theta} \\
&= \int p(y, \boldsymbol{\theta} \mid \mathbf{x}, \mathcal{D}_n) \log \frac{p(y \mid \mathbf{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta} \mid \mathcal{D}_n)}{p(y \mid \mathbf{x}, \mathcal{D}_n) p(\boldsymbol{\theta} \mid \mathcal{D}_n)} dy d\boldsymbol{\theta} \\
&= \int p(y \mid \mathbf{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta} \mid \mathcal{D}_n) \log \frac{p(y \mid \mathbf{x}, \boldsymbol{\theta})}{p(y \mid \mathbf{x}, \mathcal{D}_n)} dy d\boldsymbol{\theta} \\
&= \mathbb{E}_{\boldsymbol{\theta} \mid \mathcal{D}_n} \left[\mathbb{E}_{y \mid \mathbf{x}, \boldsymbol{\theta}} \left(\log \frac{p(y \mid \mathbf{x}, \boldsymbol{\theta})}{p(y \mid \mathbf{x}, \mathcal{D}_n)} \right) \right] \\
&= \mathbf{H}[y \mid \mathbf{x}, \mathcal{D}_n] - \mathbb{E}_{\boldsymbol{\theta} \mid \mathcal{D}_n} [\mathbf{H}[y \mid \mathbf{x}, \boldsymbol{\theta}]].
\end{aligned}$$

Hence the BALD score decomposes into predictive entropy minus the expected aleatoric–data–conditional–entropy. In Gaussian regression case, we can replace the discrete entropy by the differential entropy of a predictive Gaussian, yielding

$$\alpha_{\text{BALD}}^{\text{reg}}(\mathbf{x}) = \frac{1}{2} \log \det(\boldsymbol{\Sigma}_{\text{pred}}(\mathbf{x})) - \mathbb{E}_{\boldsymbol{\theta}} \left[\frac{1}{2} \log \sigma^2(\mathbf{x}; \boldsymbol{\theta}) \right]. \quad (2.25)$$

2.5 Memorised Variational Bayes

Classical full-batch variational Bayes (VB) revisits the entire data set at every coordinate-ascent iteration, while stochastic VB trades memory for noisy gradients and delicate learning-rate schedules. Memoised Variational Bayes (memoVB) [31, 32] offers a middle ground: it

- stores batch-level sufficient statistics in memory and
- guarantees an increase of the global VB free energy every time a batch is revisited without step-size tuning.

2.5.1 Running example: DP mixture of exponentials

We use a Dirichlet–process mixture model (DPMM) of exponential-family emissions, but the memoVB machinery applies to any model whose free energy can be written in terms of finite-dimensional sufficient statistics.

Generative model

$$\begin{aligned}
\beta_k &\sim \text{Beta}(1, \alpha_0), \\
\boldsymbol{\pi}_k &= \beta_k \prod_{l=1}^{k-1} (1 - \beta_l), \\
\theta_k^* \mid G_0 &\stackrel{iid}{\sim} G_0(\tau_0, \nu_0), \\
G &= \sum_{k=1}^{\infty} \boldsymbol{\pi}_k \delta_{\theta_k^*}, \\
s_i \mid \boldsymbol{\pi} &\sim \text{Cat}(\boldsymbol{\pi}), \\
x_i \mid s_i, \{\theta_k^*\} &\sim F(x_i \mid \theta_{s_i}^*).
\end{aligned}$$

Mean-field variational family With the usual nested truncation to K components [33], let

$$\begin{aligned} q_i(s_i) &= \text{Cat}(r_{i1}, \dots, r_{iK}), & q_k(\beta_k) &= \text{Beta}(\hat{\alpha}_{k1}, \hat{\alpha}_{k0}), \\ q_k(\theta_k^*) &= G_0(\hat{\tau}_k, \hat{\nu}_k). \end{aligned} \quad (2.26)$$

2.5.2 ELBO in terms of sufficient statistics

DEFINITION 6 (Exponential Family). *If probability distribution function can be written as*

$$p(\mathbf{x} \mid \theta) = f(\mathbf{x}) \exp(\phi(\theta)^\top \mathbf{T}(\mathbf{x}) - \mathbf{A}(\theta)), \quad (2.27)$$

then the distribution belongs to exponential family, where $\mathbf{T}$ is the sufficient statistics, ϕ is natural parameter, $\mathbf{A}$ is log normaliser and $f(\mathbf{x})$ is the base measure.

REMARK 7. *We can get expectation and variance of sufficient statistics by taking first and second derivative of log normaliser w.r.t. natural parameter.*

$$\nabla_\phi \mathbf{A}(\theta) = \mathbb{E}[\mathbf{T}(\mathbf{x})], \quad \nabla_\phi^2 \mathbf{A}(\theta) = \text{Cov}(\mathbf{T}(\mathbf{x}))$$

DEFINITION 8 (Conjugate Prior). *In Bayesian probability theory, if given a likelihood function $p(x \mid \theta)$, the posterior distribution $p(\theta \mid x)$ is in the same probability distribution family as the prior probability distribution $p(\theta)$, the prior and posterior are then called conjugate distributions and the prior is called a conjugate prior for the likelihood function $p(x \mid \theta)$.*

For exponential family, the conjugate prior can be written as

$$p(\theta \mid \tau, \nu) = F(\tau, \nu) \exp(\phi(\theta)^\top \tau - \nu \mathbf{A}(\theta)) \quad (2.28)$$

with normaliser $F(\tau, \nu)$.

REMARK 9. *The posterior distribution given conjugate prior is*

$$p(\theta \mid \{\mathbf{x}_i\}) \propto \exp\left(\phi(\theta)^\top \left(\tau + \sum_i \mathbf{T}(\mathbf{x}_i)\right) - (\nu + n) \mathbf{A}(\theta)\right),$$

with normaliser $F(\tau + \sum_i \mathbf{T}(\mathbf{x}_i), \nu + n)$.

To derive ELBO we have

$$\begin{aligned} \log p(\mathcal{X}) &= \log \int \int \sum_{s_1, \dots, s_N} \prod_{i=1}^N p(\mathbf{x}_i \mid s_i, \{\theta_k^*\}) p(s_i \mid \beta) p(\theta^*) p(\beta) d\theta^* d\beta \\ &\geq \int \int \sum_{s_1, \dots, s_N} \log \frac{p(\mathcal{X}, \mathcal{S}, \theta^*, \beta)}{\prod_{i=1}^N q_i(s_i) \prod_{k=1}^K q_k(\theta_k^*) q_k(\beta_k)} \prod_{i=1}^N q_i(s_i) \prod_{k=1}^K q_k(\theta_k^*) q_k(\beta_k) d\theta^* d\beta \\ &= \mathbb{E}_q[\log p(\mathcal{X}, \mathcal{S}, \theta^*, \beta)] + \mathbf{H}[q]. \end{aligned}$$

So we have E-like step:

$$q_i^*(s_i) \propto \exp\left(\mathbb{E}_{\prod_{\neg i} q_j(s_j) \prod_k q(\theta_k^*) q(\beta_k)} [\log p(\mathcal{X}, \mathcal{S}, \theta^*, \beta)]\right),$$

M-like step:

$$q_k^*(\theta_k^*) \propto \exp \left(\mathbb{E}_{\prod q_i(s_i) \prod_{-k} q_m(\theta_m^*) \prod q_k(\beta_k)} [\log p(\mathcal{X}, \mathcal{S}, \theta^*, \beta)] \right),$$

$$q_k^*(\beta_k) \propto \exp \left(\mathbb{E}_{\prod q_i(s_i) \prod q_k(\theta_k^*) \prod_{-k} q_m(\beta_m)} [\log p(\mathcal{X}, \mathcal{S}, \theta^*, \beta)] \right).$$

We assume G_0 and F are from exponential family and G_0 is the conjugate prior of F

$$G_0(\theta^* \mid \tau_0, \nu_0) = F(\tau_0, \nu_0) \exp \left(\phi(\theta^*)^\top \tau_0 - \nu_0 \mathbf{A}(\theta^*) \right),$$

and

$$F(\mathbf{x} \mid \theta^*) = f(\mathbf{x}) \exp \left(\phi(\theta^*)^\top \mathbf{T}(\mathbf{x}) - \mathbf{A}(\theta^*) \right).$$

We have

$$\begin{aligned} \sum_{i=1}^N \log (p(\mathbf{x}_i \mid s_i, \{\theta_k^*\}) p(s_i \mid \beta)) &= \sum_{i=1}^N \log (\pi_{s_i} F(\mathbf{x}_i \mid \theta_{s_i}^*)) \\ &= \sum_{i=1}^N \sum_{k=1}^K \delta_k(s_i) (\log \pi_k + \log F(\mathbf{x}_i \mid \theta_k^*)) \\ &= \sum_{k=1}^K N_k \log \pi_k + \sum_{k=1}^K \sum_{i=1}^N \delta_k(s_i) \log F(\mathbf{x}_i \mid \theta_k^*) \\ &= \sum_{k=1}^K N_k (\log \pi_k - \mathbf{A}(\theta_k^*)) + \phi(\theta_k^*)^\top \mathbf{S}_k + \dots \end{aligned}$$

where $N_k = \sum_{i=1}^N \delta_k(s_i)$ and $\mathbf{S}_k = \sum_{i=1}^N \delta_k(s_i) \mathbf{T}(\mathbf{x}_i)$. So

$$\begin{aligned} \log p(\mathcal{X}, \mathcal{S}, \theta^*, \beta) &= \sum_{k=1}^K N_k (\log \pi_k - \mathbf{A}(\theta_k^*)) + \phi(\theta_k^*)^\top \mathbf{S}_k + \phi(\theta_k^*)^\top \tau_0 - \nu_0 \mathbf{A}(\theta_k^*) + (\alpha_0 - 1) \log(1 - \beta_k) + \dots \\ &= \sum_{k=1}^K \left(N_k \log \beta_k + \left(\sum_{j>k}^K N_j + \alpha_0 - 1 \right) \log(1 - \beta_k) + \phi(\theta_k^*)^\top (\mathbf{S}_k + \tau_0) - (N_k + \nu_0) \mathbf{A}(\theta_k^*) \right) + \dots \end{aligned} \tag{2.29}$$

We define

$$\mathbb{E}_{q(s)} [N_k] = \mathbb{E}_{q(s)} \left[\sum_{i=1}^N \delta_k(s_i) \right] = \sum_{i=1}^N r_{ik} =: \hat{n}_k,$$

and

$$\mathbb{E}_{q(s)} [\mathbf{S}_k] = \mathbb{E}_{q(s)} \left[\sum_{i=1}^N \delta_k(s_i) \mathbf{T}(\mathbf{x}_i) \right] = \sum_{i=1}^N r_{ik} \mathbf{T}(\mathbf{x}_i) =: \hat{\mathbf{s}}_k$$

and per-batch counterparts $\hat{n}_{bk}$, $\hat{\mathbf{s}}_{bk}$ for batch $b = 1, \dots, B$.

2.5.3 Parameter Updates

From (2.29) we can update variational approximation distribution defined in (2.26)

$$\begin{aligned}\hat{\alpha}_{k1} &= 1 + \hat{n}_k, & \hat{\alpha}_{k0} &= \alpha_0 + \sum_{j>k}^K \hat{n}_j, \\ \hat{\tau}_k &= \tau_0 + \hat{\mathbf{s}}_k, & \hat{\nu}_k &= \nu_0 + \hat{n}_k.\end{aligned}\tag{2.30}$$

We also have

$$\mathbb{E}_{q(\beta)} [\log \pi_k] = \psi(\hat{\alpha}_{k1}) - \psi(\hat{\alpha}_{k1} + \hat{\alpha}_{k0}) + \sum_{l<k} [\psi(\hat{\alpha}_{l0}) - \psi(\hat{\alpha}_{l1} + \hat{\alpha}_{l0})].$$

Local responsibilities for data point $\mathbf{x}_i$ follow

$$q_i(s_i = k) = r_{ik} \propto \exp \left(\mathbb{E}_{q(\beta)} [\log \pi_k] + \mathbb{E}_{q(\theta_k^*)} [\log F(\mathbf{x}_i | \theta_k^*)] \right) \tag{2.31}$$

so r_{ik} can be calculated using softmax function over k with logit $\mathbb{E}_{q(\beta)} [\log \pi_k] + \mathbb{E}_{q(\theta_k^*)} [\log F(\mathbf{x}_i | \theta_k^*)]$.

2.5.4 MemoVB: cache-update-replace loop

Partition the data once into fixed batches $\{\mathcal{B}_b\}_{b=1}^B$ and cache their statistics.

- Remove old batch summary.

$$\hat{n}_k \leftarrow \hat{n}_k - \hat{n}_{bk}, \quad \hat{\mathbf{s}}_k \leftarrow \hat{\mathbf{s}}_k - \hat{\mathbf{s}}_{bk}.$$

- Recompute locals for $\mathcal{B}_b$ via the standard r_{nk} formula; obtain new $\hat{n}_{bk}^{\text{new}}, \hat{\mathbf{s}}_{bk}^{\text{new}}$.
- Add the new summary.

$$\hat{n}_k \leftarrow \hat{n}_k + \hat{n}_{bk}^{\text{new}}, \quad \hat{\mathbf{s}}_k \leftarrow \hat{\mathbf{s}}_k + \hat{\mathbf{s}}_{bk}^{\text{new}}.$$

- Update global factors with the refreshed $(\hat{n}_k, \hat{\mathbf{s}}_k)$.

Because each step is an exact coordinate-ascent move on the free energy objective,

$$\mathcal{F}_{\text{after batch}} \geq \mathcal{F}_{\text{before batch}},$$

hence the full-data free energy monotonically increases while processing only one mini-batch in memory.

2.5.5 Complexity and advantages

- **Time.** Same per-sample complexity as stochastic VB ($\mathcal{O}(|\mathcal{B}_b| K)$ for a DPMM), but no noisy gradients.
- **Memory.** Store $B \times K$ summaries, not the raw data.

- **Robustness.** No learning-rate schedule; each update is guaranteed to improve or leave unchanged the free energy.
- **Escaping local optima.** [34, 35, 32, 36, 37] introduce birth and merge moves that propose component-splits using a small targeted subset, accepting the move only if it raises the global free energy, again exploiting the cached summaries.

REMARK 10. When $B = 1$ memoVB degenerates to full-batch VB; when $B = N$ (one data point per batch) it resembles incremental EM [38]. Thus, memoVB forms a continuum between batch and stochastic inference.

2.6 Reinforcement Learning

2.6.1 Markov Decision Processes (MDPs)

DEFINITION 11. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ where

- $\mathcal{S}$ is the set of all possible states,
- $\mathcal{A}$ is the set of all possible actions,
- $p(s' \mid s, a) = p(s_{t+1} = s' \mid s_t = s, a_t = a)$ is the probability of transitioning to s' , given a state s and action a ,
- $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the expected reward, achieved on a transition starting in (s, a) , i.e. $r(s, a) = \mathbb{E}[R_t \mid s_t = s, a_t = a]$. R_t is the immediate reward.
- $\gamma \in [0, 1]$ is a discount factor that trades off later rewards to earlier ones.

We define returns G_t such that

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t} R_T = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

The goal of an RL agent is to find a behaviour policy that maximises the expected return G_t . A policy π is a mapping $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ that, for every state s , assigns for each action $a \in \mathcal{A}$ the probability of taking that action in state s , denoted by $\pi(a \mid s)$.

We also define value function $V_t^\pi(s)$ and state-action value $Q_t^\pi(s, a)$:

$$V_t^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \quad (2.32)$$

$$Q_t^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]. \quad (2.33)$$

DEFINITION 12 (Optimal Value Functions). The optimal value function $V^*(s)$ is the maximum value function over all policies

$$V^*(s) = \max_{\pi} V^\pi(s).$$

The optimal action-value function $Q^*(s, a)$ is the maximum action-value function over all policies

$$Q^*(s, a) = \max_{\pi} Q^\pi(s, a).$$

THEOREM 13 (Optimal Policies). *For any MDP, there exists an optimal policy π^* that is better than or equal to all other policies, $\pi^* \geq \pi, \forall \pi$. All optimal policies achieve the optimal value function, $V^{\pi^*}(s) = V^*(s)$. All optimal policies achieve the optimal action-value function, $Q^{\pi^*}(s, a) = Q^*(s, a)$.*

Value functions can be defined recursively, which is known as Bellman Equations.

THEOREM 14 (Bellman Expectation Equations). *Given an MDP, $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, p, r, \gamma \rangle$, for any policy π , the value functions obey the following expectation equations:*

$$V^\pi(s) = \sum_a \pi(s, a) \left[r(s, a) + \gamma \sum_{s'} p(s' | a, s) V^\pi(s') \right] \quad (2.34)$$

$$Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | a, s) \sum_{a' \in \mathcal{A}} \pi(a' | s') Q^\pi(s', a'). \quad (2.35)$$

Similarly we have Bellman Optimal Equations.

THEOREM 15 (Bellman Optimality Equations). *Given an MDP, $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, p, r, \gamma \rangle$, for any policy π , the optimal value functions obey the following expectation equations:*

$$V^*(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s' | a, s) V^*(s') \right] \quad (2.36)$$

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | a, s) \max_{a' \in \mathcal{A}} Q^*(s', a'). \quad (2.37)$$

2.6.2 Partially Observable MDPs (POMDPs)

A partially observable Markov decision process (POMDP) is a generalisation of a Markov decision process. A POMDP models an agent decision process in which it is assumed that the system dynamics are determined by an MDP, but the agent cannot directly observe the underlying state.

DEFINITION 16. *A POMDP is a 7-tuple $(\mathcal{S}, \mathcal{A}, p, r, \Omega, O, \gamma)$. In addition to MDP,*

- Ω is the set of observations,
- $O(o | s') = p(o_{t+1} = o | s_{t+1} = s')$

The agent maintains a belief state that the environment may (or not) be in.

DEFINITION 17 (Belief State). *Let b be a probability distribution over the state space $\mathcal{S}$. $b(s)$ denotes the probability that the environment is in state s .*

$$b_t(s) = p(s_t = s | o_{1:t}, a_{1:t-1}).$$

After taking action a_t and get observation o_{t+1} , by Bayes' rule,

$$b_{t+1}(s') = \frac{O(o_{t+1} | s') \sum_s P(s' | s, a_t) b_t(s)}{\sum_{s'} O(o_{t+1} | s') \sum_s P(s' | s, a_t) b_t(s)} \propto O(o_{t+1} | s') \sum_s P(s' | s, a_t) b_t(s)$$

2.6.3 Value-based Learning

Value-based learning focuses on algorithms to approximate Q^* (or V^*) and derive π by greedy action selection. Value-based learning does not require the knowledge of MDP, only requires samples. It is model free. There are 2 types of methods: Monte-Carlo and temporal-difference (TD) learning. They are based on Bellman equations, i.e.

$$V^\pi(s) = \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s].$$

- Monte Carlo: update value $V^\pi(S_t)$ towards sampled return G_t ,

$$V^\pi(S_t) \leftarrow V^\pi(S_t) + \alpha(G_t - V^\pi(S_t)).$$

- Temporal-difference learning: update value $V^\pi(S_t)$ towards estimated return $R_{t+1} + \gamma V^\pi(S_{t+1})$,

$$V^\pi(S_t) \leftarrow V^\pi(S_t) + \alpha(R_{t+1} + \gamma V^\pi(S_{t+1}) - V^\pi(S_t)),$$

where $\delta_t = R_{t+1} + \gamma V_t^\pi(S_{t+1}) - V_t^\pi(S_t)$ is called the TD error.

One important aspect of reinforcement learning is on/off policy learning. On-policy methods are about learning from what the agent is currently doing. The agent learns based on the actions it is currently taking. Off-policy, on the other hand, learns about hypothetical, counterfactual events (i.e. “what if” questions).

2.6.4 Policy-based Learning

Policy-based learning optimises a parametrised policy π_θ directly. It can be easily extended to high-dimensional or continuous action spaces and can learn stochastic policies. It also has advantages when policies are simple but values or models are complex.

In policy-based learning, the target can be average total return per episode in episodic environments, or average reward per step in continuing environments.

- Episodic-return objective:

$$\begin{aligned} J(\theta) &= \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] \\ &= \mathbb{E}_{\tau \sim p_\theta} [G_0] \\ &= \mathbb{E}_{S_0 \sim d_0} [\mathbb{E}[G_t \mid S_t = S_0]] \\ &= \mathbb{E}_{S_0 \sim d_0} [V^\pi(S_0)] \end{aligned} \tag{2.38}$$

where $p_\theta(\tau) = p(s_0) \prod_t \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$ is the distribution of trajectory and d_0 is the distribution of S_0 .

- Average-reward objective:

$$\begin{aligned} J(\theta) &= \mathbb{E}[R_{t+1}] \\ &= \mathbb{E}_{S_t \sim d_{\pi_\theta}} [\mathbb{E}_{A_t \sim \pi_\theta(S_t)} [R_{t+1} \mid S_t]] \end{aligned} \tag{2.39}$$

where $d_\pi(s) = p(S_t = s \mid \pi)$ is the probability of being in state s in the long run.

THEOREM 18 (Policy Gradient Theorem (Episodic) [39]). *For any differentiable policy $\pi_\theta(a \mid s)$, let d_0 be the starting distribution over states in which we begin an episode. Then, the policy gradient of $J(\theta) = \mathbb{E}_{\tau \sim p_\theta} [G_0]$ is*

$$\nabla_\theta J(\theta) = \mathbb{E}_{d_0, \pi_\theta} \left[\sum_{t=0}^T \gamma^t Q^{\pi_\theta}(S_t, A_t) \nabla_\theta \log \pi_\theta(A_t \mid S_t) \right], \quad (2.40)$$

where

$$Q^{\pi_\theta}(s, a) = \mathbb{E}_{\pi_\theta} [G_t \mid S_t = s, A_t = a] = \mathbb{E}_{\pi_\theta} [R_{t+1} + \gamma Q^{\pi_\theta}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

If we ignore γ^t term, we can have the following update:

REINFORCE (Monte-Carlo)

$$\Delta\theta_t = \alpha G_t \nabla_\theta \log \pi_\theta(A_t \mid S_t),$$

such that

$$\mathbb{E}_\pi \left[\sum_t \Delta\theta_t \right] = \nabla_\theta J(\theta).$$

Actor-Critic

Recall that the expectation of fisher score, evaluated at the true parameter value θ is 0, i.e.

$$\mathbb{E}_\pi [\nabla_\theta \log \pi_\theta(A_t \mid S_t)] = 0.$$

So for any $b(S_t)$ that does not depend on A_t , we have

$$\mathbb{E}_\pi [b(S_t) \nabla_\theta \log \pi_\theta(A_t \mid S_t)] = 0.$$

We can use this as baseline to reduce the variance. The common baseline is value function $V^\pi(S_t)$. Typically we estimate $V_w(s) \approx V^\pi(s)$ with some functional form or neural network. We compute one-step TD-error, or generalised advantage estimation (GAE) [40]:

$$\begin{aligned} \delta_t &= R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t), \\ \hat{A}_t^{(\lambda)} &= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \end{aligned}$$

where $\lambda \in [0, 1]$ is the trace parameter. $\lambda = 0$ reduces to TD(0) case and $\lambda = 1$ is Monte-Carlo case.

- **Actor**

$$\Delta\theta_t = \alpha A_t \nabla_\theta \log \pi_\theta(A_t \mid S_t), \quad (\text{AC-actor})$$

with A_t being one-step TD-error or GAE.

- **Critic**

$$\Delta w = -\beta \delta_t \nabla_w V_w(S_t), \quad (\text{AC-critic})$$

i.e. TD(0) regression on $V_w(s) \approx V^\pi(s)$.

Entropy Regularisation To encourage exploration we can add entropy $\eta \mathbf{H}[\pi_\theta(\cdot | S_t)]$ to the actor reward; the gradient becomes

$$\alpha A_t \nabla_\theta \log \pi_\theta(A_t | S_t) + \eta \nabla_\theta \mathbf{H}[\pi_\theta(A_t | S_t)].$$

Value-based and policy-based methods are complementary: value learners excel in discrete action spaces with off-policy data, while policy gradients handle continuous actions and stochastic policies; modern algorithms (e.g. SAC [41], DDPG [42]) blend both ideas.

Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) [20] is a policy-gradient method that improves stability by constraining the update between successive policies. We maintain an actor (policy) $\pi_\theta(A_t | S_t)$ and a critic (value function) $V_w(s)$.

Surrogate Objective Let $\pi_{\theta_{\text{old}}}$ denote the policy before the update, and define the importance sampling ratio

$$r_t(\theta) = \frac{\pi_\theta(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}. \quad (2.41)$$

The unclipped policy-gradient objective is

$$L^\pi(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right], \quad (2.42)$$

where $\hat{A}_t$ is an estimator of the advantage function.

To avoid large policy updates, PPO introduces a clipped surrogate objective:

$$L_{\text{clip}}^\pi(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (2.43)$$

with clipping parameter $\epsilon \in (0, 1)$.

Entropy Bonus To encourage exploration, an entropy regulariser is added:

$$L_{\text{ent}}^\pi(\theta) = L_{\text{clip}}^\pi(\theta) + \eta \mathbb{E}_t [\mathbf{H}(\pi_\theta(A_t | S_t))], \quad (2.44)$$

where $\mathbf{H}$ is the categorical entropy and $\eta > 0$ a weight.

Value Function Update The critic is updated by minimising the squared error between predicted values and empirical returns:

$$L^V(w) = \mathbb{E}_t \left[(V_w(S_t) - \hat{R}_t)^2 \right], \quad (2.45)$$

where the return target is $\hat{R}_t = \hat{A}_t + V_w(S_t)$.

Advantage Estimation We use Generalised Advantage Estimation (GAE) [40]:

$$\delta_t = r_t + \gamma V_w(S_{t+1})(1 - d_t) - V_w(s_t), \quad (2.46)$$

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad (2.47)$$

with discount factor $\gamma \in (0, 1)$ and trace parameter $\lambda \in [0, 1]$. Here d_t is the episode termination flag.

Final Objective The overall PPO loss combines the actor and critic terms:

$$L(\theta, w) = L_{\text{ent}}^{\pi}(\theta) - c_v L^V(w), \quad (2.48)$$

where c_v balances the critic loss against the actor objective. Gradient ascent on L_{ent}^{π} updates the policy parameters θ , and gradient descent on L^V updates the value parameters w .

2.7 NetHack: a Roguelike With Structured, Symbolic State

2.7.1 Game overview

NetHack (1987 - present) is a single-player, ASCII-rendered dungeon crawler in which the agent (“hero”) must descend up to 50 levels, retrieve the Amulet of Yendor, and ascend to the surface. Gameplay is turn-based: at every time-step t the agent chooses a single key-press $A_t \in \mathcal{A}$ (121 possible keys), after which the environment deterministically updates the dungeon and emits a textual observation. Although classic NetHack presents the screen to a human terminal, the NetHack Learning Environment (NLE) [5] and MiniHack [6] convert that output into a fixed-width tensor that is amenable to deep learning.

2.7.2 Observation tuple

Following the NLE API, each time-step produces human-visible observation

$$O_t = (G_t, C_t, S_t, M_t), \quad (2.49)$$

where the default screen size is $H = 21$, $W = 79$. All tensors are `uint8` unless stated otherwise:

- **Glyph char** $G_t \in \{32, \dots, 127\}^{H \times W}$
Printable ASCII code of the visible glyph in every cell ( hero,  boulder,  river, ...).
- **Glyph colour** $C_t \in \{0, \dots, 15\}^{H \times W}$
Sixteen ANSI colours (red=1, green=2, white=15, ...). Colours represent the status of the object. For example, a green object means it is blessed while a red one means it is cursed. Most commonly an object is shown as unknow status with white or light grey colour.

- **Basic statistics** $S_t \in \mathbb{N}^{27}$

A fixed vector of numerical fields defined by NetHack (“**blstats**”). Table A.1 lists the meaning of each index.

- **Message line** $M_t \in \{0, \dots, 127\}^L$

The last terminal line, padded to $L = 256$ bytes, e.g.

"Hello Agent, welcome to NetHack! You are a chaotic make human Rogue."

The first ever message shows the identity of the hero in this episode. A hero attributes contain role, race, gender and alignment. It is static for the entire episode. Each field is an integer index ($15 \times 5 \times 2 \times 3 = 450$ possible combinations)

Figure 2.2 shows a human-visible screen for NetHack.

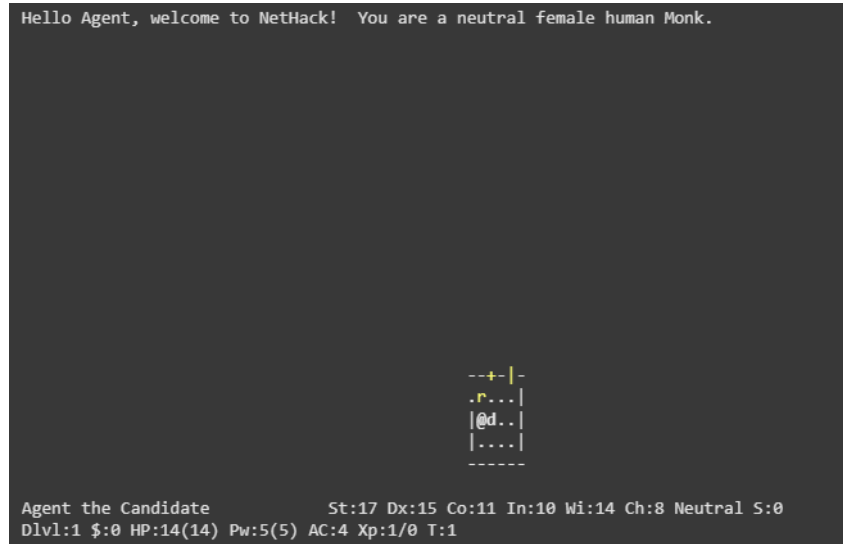


Figure 2.2: NetHack Game

It is worth noting that NLE provides **glyph** in observation keys. **Glyph** is semantic identifier which has range 0 to more than 6000. It has richer semantic information than the combination of **char** and **color**. For example, a single monster type (like “orc”) has multiple representations of glyphs.

- Normal orc (**glyph** = 100)
- Invisible orc (**glyph** = 481)
- Ridden orc (**glyph** = 2005)

glyph encodes what the object is, the state the object is in and the context. Multiple **glyph** can map to the same character and colour. Human player can never see **glyph** and the rich information. Instead they can only observe **char** and **color**. This makes NetHack a discovery game as players do not know the exact properties and they must experiment and explore to learn. Such features make NetHack a perfect game for AI training of unknown exploration and progressive learning, which is the foundation of open endedness.

2.7.3 Action space

The agent emits a single ASCII key per turn. MiniHack exposes $|\mathcal{A}| = 75$ actions (8 compass moves, 8 run moves, 8 far moves, inventory keys, spell-casting, ...). The environment resolves multi-step commands (e.g. **Shift+6** for “run east”) into one-step updates, so the action space remains stationary. For each game, MiniHack defines available actions by default. For example, for navigation task, 8 compass moves are used as action space.

2.7.4 Goal

The long-term primary goal of NetHack is to retrieve the Amulet of Yendor from the deepest level of the dungeon and offer it to deity on the Astral Plane to “ascend”. Along the way, the hero need to descend through 50+ randomized dungeon levels filled with traps, monsters and treasures, collect powerful artifacts necessary for entering into Astral Plane. The hero needs to manage hunger, HP and status effects and balance the risk and reward. After obtaining the Amulet, the hero needs to navigate to the Plane of Earth, Water, Fire and finally Astral Plane. Then the hero offers the Amulet at the altar to the god and if they approve, the hero can ascend and win the game.

In NLE and MiniHack, things are easier as each dungeon is an episode. The hero gets the reward and episode stops when they reach the staircase to next level  if still descending, or obtain the Amulet.

Chapter 3

Methodology

In this chapter we will go through the methodology of playing NetHack/MiniHack. NetHack/MiniHack observations x_t are high-dimensional (21×79 glyphs, colours, blstats and messages). It has sparse reward — MiniHack emits 0 for valid action, -0.01 for invalid action or bumping into walls and $+1$ reward only when reaching the goal. The competent play demands temporally extended “skills” (e.g., explore a room, approach stairs, push boulder into water to make a way) that last many steps. These skills are discrete in nature (either the agent is “in corridor traversal” or not), exhibit temporal persistence (once engaged, they last many steps) and involve sequential dependencies (some skills naturally follow other: e.g. “open door” $\rightarrow$ “enter room”). From a modelling perspective, this implies we need

- a continuous latent state z_t that encodes the fine-grained geometry and semantics of each observation for accurate prediction and reconstruction. This can be provided by variational auto-encoder;
- a discreet latent skill variable h_t that evolves slowly, clusters z_t into macro-states, and preserves sequential structure;
- a downstream policy architecture to consume latent space z_t and belief of the latent skill variable h_t based on current latent variable. The reward structure needs enrichment to remediate the effect of sparse reward.

The hierarchy of methodology summarises in Fig 3.1¹, specifically:

- A latent world model will be trained with Variational Autoencoder. The world model has 2 heads — reconstruction $p_\theta(x_t | z_t)$ and dynamic world model $p_\theta(z_{t+1} | z_t, a_t, \alpha_t)$.
- A skill latent space which links the latent encoding space and low-level action space. The skill latent space is modelled with sticky hierarchical Dirichlet process with hidden Markov model (sticky-HDP-HMM [19]).
- An PPO policy with actor-critic network with latent variable z_t and skill belief α_t with curiosity-driven rewards.

¹Codebase in <https://github.com/XuChenCatkin/SequentialSkillRL>

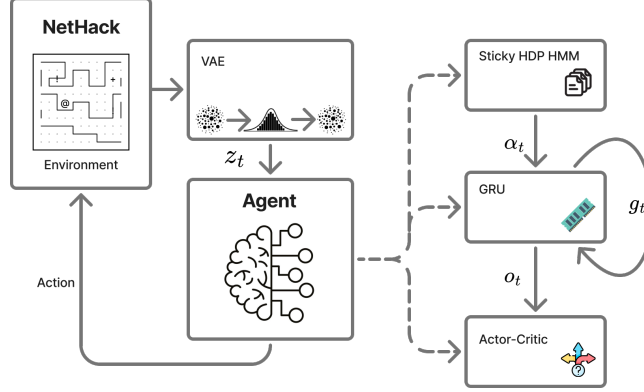


Figure 3.1: Three-layer Online Learning Hierarchy for NetHack/MiniHack Game Play

3.1 Multi-Modal Variational Autoencoder for MiniHack

We design a single VAE that jointly models the heterogeneous observation channels of MiniHack—glyph characters, glyph colours, game statistics, text messages, and hero metadata—and compresses them into a continuous latent $z_t \in \mathbb{R}^D$. The encoder outputs a Gaussian posterior with a diagonal + low-rank covariance, which improves modelling capacity without the cost of a full covariance matrix. The decoder reconstructs both global map information and ego-centric local targets relevant for control.

3.1.1 Encoder and Decoder Architectures

Map encoder We embed the glyph character IDs (ASCII 32–127) and colour IDs (0–15), concatenate a foreground mask and two CoordConv channels $(x, y) \in [-1, 1]$, then process the 21×79 grid with a CNN backbone that mixes standard and anisotropic dilations (wider receptive fields along the map width). We aggregate spatial features via masked attention pooling with 2 learned queries [43], passing to layer norm [44] and yielding a fixed-length map vector.

Stats encoder Raw `blstats` are a 27-length integer vector with detailed descriptions in A.1. We preprocessed raw stats by

- mapping `x` and `y` coordination into $[0, 1]$ with width and height of the map,
- applying $\log(1 + x)$ to `score`, `gold` and `experience_points` to avoid numeric explosion,
- converting `hitpoints` and `energy` to ratios provided with their max,
- embedding `hunger_state`, `dungeon_number` and `level_number`,

- decomposing `condition_mask` to conditions by applying bitwise operation and put them into one hot vector.

The raw `blstats` is preprocessed into a 43-dimensional vector and fed to a small MLP to produce a compact feature.

Message encoder The tokenized message (length 256) is embedded and passed to a bi-GRU; we project the concatenated hidden states to a fixed-size message vector.

Hero embedding. We embed `role`, `race`, `gender`, `alignment` and concatenate them into a small fixed vector.

Fusion and posterior heads We concatenate the map, stats, message, and hero vectors and pass them through an MLP to produce the VAE heads:

$$\mu_\phi(x_t) \in \mathbb{R}^D, \quad \log \sigma_\phi^2(x_t) \in \mathbb{R}^D, \quad F_\phi(x_t) \in \mathbb{R}^{D \times R},$$

where F_ϕ are optional low-rank factors (rank R). The posterior covariance is

$$\Sigma_q(x_t) = \text{diag}(\sigma_\phi^2(x_t)) + F_\phi(x_t) F_\phi(x_t)^\top.$$

We sample z_t via the reparameterisation trick.

Decoders and reconstruction targets Given z_t , we decode the following (each with its appropriate likelihood):

- **Global occupancy and glyph bag:** pixel-wise foreground occupancy with binary cross entropy loss and a bag-of-glyphs histogram for character $\times$ colour pairs with multinomial cross entropy loss. The bag term forces latent space to encode global appearance statistics.
- **Ego-centric patch:** FiLM [45] layers with Conv2d and coordinates to emit character, colour logits on a $k \times k$ window centred at the agent. We also categorised the character $\times$ colour pair into 17 semantic class. This can help VAE encode and learn the semantics of ego-view which is helpful for action decision. The whole semantic class is listed in A.2. From the class logits we derive passibility and safety distributions in the 8 neighbouring directions with binary cross entropy loss.
- **Stats:** continuous components with Gaussian negative loglikelihood loss (mean / log-variance heads), and discrete indices (hunger tier, dungeon branch, level) with cross entropy loss; condition bits with binary cross entropy loss.
- **Message:** Conv1d to decode token sequence with masked cross-entropy that ignores PAD and all positions after the first EOS.
- **Hero presence/centroid:** a head predicts hero present with binary cross entropy and its centroid (mean squared error loss, only applied when present), grounding spatial semantics.

- **Action auxiliary head:** an auxiliary decoder head that predicts local movement affordances around the agent (passibility and safety in several directions) and a goal direction signal. It takes the logits from ego semantic class and multiply semantic class to passibility / safety static mapping to get the prior logits for passibility and safety. We also apply a small MLP to map latent variable z_t to latent-gated correction delta. The predicted passibility and safety logits are formed by summing prior logits and gated correction delta. We also apply a small MLP to map z_t to goal coordinates. The goal is usually the nearest staircase. These auxiliary losses provide an action-relevant training signal that shapes the latent z_t toward features useful for navigation / policy learning.
- **World model:** the world model head equips VAE with predictive coding. It predicts the next latent state z_{t+1} , the extrinsic reward r_t , and the termination flag d_t given current latent z_t and action a_t . This encourages the latent space to be dynamically meaningful for reinforcement learning, rather than purely reconstructive. The next latent state and extrinsic reward use square loss. The termination flag use binary cross entropy loss.
- **Inverse dynamics:** predicts the action a_t that caused the transition (z_t, z_{t+1}) . It ensures the latent captures controllable aspects of the environment, i.e. features relevant for agent’s interventions. It complements the world model as auxiliary pair for representation learning in RL.

3.1.2 Loss: Multi-term Reconstruction + β -Decomposed KL

Let $q_\phi(z_t | x_t) = \mathcal{N}(\mu_\phi, \Sigma_q)$ with $\Sigma_q = \text{diag}(\sigma_\phi^2) + F_\phi F_\phi^\top$. The total loss for VAE is

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q_\phi(z_t | x_t)} [\mathcal{L}_{\text{recon}}(x_t, \hat{x}_t; \theta, \phi) + \mathcal{L}_{\text{aux}}(z_t)] + \beta D_{\text{KL}}[q_\phi(z_t | x_t) \| p(z_t)],$$

KL term and priors We support three priors:

1. **Standard:** $p(z) = \mathcal{N}(0, I)$, with the closed-form $D_{\text{KL}}(q \| p)$ for diagonal+low-rank Σ_q . Using the matrix determinant lemma and Woodbury identity, we have

$$\log \det \Sigma_q = \sum_i \log \sigma_{\phi, i}^2 + \log \det \left(I_r + F_\phi^\top \text{diag}(\sigma_\phi^{-2}) F_\phi \right),$$

and $\text{tr}(\Sigma_q)$ analogously, then uses the standard Gaussian KL.

2. **Sticky HDP-HMM prior (Sec: 3.2):** $p(z_t) = \sum_k \hat{r}_k \mathcal{N}(\hat{\mu}_k, \hat{\Lambda}_k^{-1})$ where $(\hat{\mu}_k, \hat{\Lambda}_k^{-1})$ are the NIW posteriors from the skill model. We have

$$D_{\text{KL}}[q_\phi(z_t) \| p(z_t | h_t=k)] \quad \text{for all } k, \text{ and then take } \mathbb{E}_{r_{t,k}}[\cdot]$$

under the smoothing responsibilities $\hat{r}_{t,k}$ supplied in the batch.

3. **Blended prior:** a convex blend $\alpha \cdot \text{HMM} + (1-\alpha) \cdot \mathcal{N}(0, I)$.

To avoid posterior collapse in VAE training ($\text{KL} \rightarrow 0$ and the decoder ignoring z), we would like to utilise β -TC-VAE decomposition and free bits.

Decomposition into MI / TC / DW Following the β -TC-VAE decomposition [46],

$$\begin{aligned}\mathbb{E}_{p(x)} [D_{\text{KL}}[q(z|x) \| p(z)]] &= \text{MI} + \text{TC} + \text{DWKL} \\ &= D_{\text{KL}}[q(z, x) \| q(z)q(x)] + D_{\text{KL}} \left[q(z) \| \prod_j^D q(z_j) \right] + \sum_j D_{\text{KL}}[q(z_j) \| p(z_j)],\end{aligned}$$

where MI is the mutual information between x and z , TC is the Total Correlation (TC) of $q(z)$ (penalizing correlated latents), and DWKL is the dimension-wise KL. We estimate the three pieces from minibatch statistics:

- **Within-posterior covariance:** $\bar{\Sigma}_q = \mathbb{E}_{p(x)}[\Sigma_q(x)]$, computed exactly for the diagonal+low-rank form.
- **Between-mean covariance:** $S_\mu = \text{Cov}_{p(x)}[\mu_\phi(x)]$.
- **Total latent covariance:** $S = \bar{\Sigma}_q + S_\mu$.

Let $\text{diag}(S)$ be the vector of marginal variances and define the correlation matrix $R = \text{diag}(S)^{-1/2} S \text{diag}(S)^{-1/2}$. Then

$$\text{TC} = -\frac{1}{2} \log \det R, \quad \text{DWKL} = \frac{1}{2} \sum_{i=1}^D \left(S_{ii} + (\bar{\mu}_i - \mu_{p,i})^2 - 1 - \log S_{ii} \right),$$

where $\bar{\mu}$ is the batch mean of $\mu_\phi(x)$ and μ_p is the mean of prior. Finally, $\text{MI} = \text{KL} - \text{TC} - \text{DWKL}$. We can apply different β for the decompositions. We do not want to penalise mutual information as it encourages retention of the input information in the latent space and avoid posterior collapse. Total correlation controls disentanglement and prevents redundancy across latent dimensions, promoting factorized representations. Hence TC is heavily penalised as we would like the latent dimensions to encode a distinct factor. DWKL enforces marginal alignment with prior; it is mainly a regulariser and we can set a moderate penalty. This selective reweighting allows the latent space to balance informativeness, disentanglement, and Bayesian consistency, which is crucial for sequential skill preservation in NetHack.

Free bits (per-dimension) In addition to β -TC-VAE, we apply a free-bits [47] threshold λ per latent dimension to the DWKL term:

$$\text{DWKL}^{(\lambda)} = \sum_{i=1}^D \max\{\text{DWKL}_i - \lambda, 0\}.$$

This guarantees that a small, unavoidable KL budget is “free” and does not get punished, encouraging the posterior to use more latent capacity.

3.1.3 β -Annealing and Stability

The three β weights— $\beta_{\text{MI}}, \beta_{\text{TC}}, \beta_{\text{DW}}$ —are annealed from small initial values to their targets early in training. Practically, this: (i) lets the decoder learn easy reconstructions first, (ii) prevents the optimizer from immediately crushing KL to zero, and (iii) gives the low-rank covariance time to

stabilize. We also keep all reconstruction heads active from the beginning so the latent is pressured to encode spatial, textual, and scalar content jointly.

3.2 Skill Space via a Sticky Hierarchical DP HMM

Instead of placing standard normal prior on latent space z_t , we would like to construct a discrete skill space $\{h_t\}_{t=1}^T$ whose emission is Gaussian. This forms piece-wise-stationary Gaussian clusters of encoder latent. Each cluster is a macro-action lasting a few dozen of steps. The Gaussian cluster can be modelled with Dirichlet process (2.13). With a hierarchical Dirichlet process prior, the model can expand the skill set online (2.8), which matches the open-ended discovery in NetHack/MiniHack. In addition, a Hidden Markov Model 2.3 is the natural Bayesian machinery for temporal dependence $p(h_t | h_{t-1})$ and its Markov structure that provides exactly the kind of skill transition graph we want for sequential plans. Following [19] we place a sticky hierarchical Dirichlet-process hidden Markov model (sticky-HDP-HMM) on top of the continuous VAE latents $z_{1:T} = \{z_t\}_{t=1}^T$. The construction yields an unbounded library of temporally persistent skills, indexed by a discrete hidden state $h_t \in \{1, 2, \dots\}$.

Generative hierarchy. Inspired by (2.13), we use conventional notations in HMM and we have global stick-breaking construction for stick weights π . Instead of having i.i.d. cluster indices, we create DPs on state-specific transition. Mathematically,

$$\begin{aligned}
\beta_k &\sim \text{Beta}(1, \gamma), \\
\pi &\sim \text{GEM}(\gamma), \pi_k = \beta_k \prod_{l < k} (1 - \beta_l) \quad (\text{global stick-breaking weights}) \\
h_1 &\sim \text{Cat}(\pi), \quad (\text{initial HMM state}) \\
\Phi_k &\sim \text{DP}\left(\alpha + \kappa, \frac{\alpha}{\alpha + \kappa} \pi + \frac{\kappa}{\alpha + \kappa} \delta_k\right), \quad (\text{row } k \text{ of transition matrix } \Phi) \\
h_t | h_{t-1} &\sim \text{Cat}(\Phi_{h_{t-1}}), \quad (\text{sticky Markov chain})
\end{aligned} \tag{3.1}$$

where α is the concentration, $\kappa > 0$ the stickiness bias, and γ the top-level DP mass. Setting $\kappa > 0$ inflates the diagonal of every transition row, encouraging dwell times longer than one step:

$$\begin{bmatrix} \Phi_{11} & \dots & \Phi_{1K} & \dots \\ \vdots & \ddots & \vdots & \vdots \\ \Phi_{K1} & \dots & \Phi_{KK} & \dots \\ \vdots & \dots & \dots & \dots \end{bmatrix} \sim \begin{bmatrix} \text{Dir}(\alpha\pi_1 + \kappa, \dots, \alpha\pi_{k-1}, \alpha\pi_k, \alpha\pi_{k+1}, \dots) \\ \vdots \\ \text{Dir}(\alpha\pi_1, \dots, \alpha\pi_{k-1}, \alpha\pi_k + \kappa, \alpha\pi_{k+1}, \dots) \\ \vdots \end{bmatrix}$$

For the emission parameter, we place same Gaussian emission prior across different clusters.

$$\begin{aligned}
\eta_k &= (\mu_k, \Lambda_k^{-1}) \sim \text{NIW}(\mu_0, \kappa_0, \Psi_0, \nu_0), \quad (\text{NIW prior}) \\
z_t | h_t &\sim \mathcal{N}(\mu_{h_t}, \Lambda_{h_t}^{-1}), \quad (\text{Gaussian emission})
\end{aligned} \tag{3.2}$$

where NIW is Normal-inverse-Wishart distribution, which is the conjugate prior for Normal distribution with unknown mean and covariance matrix.

As we have shown in (2.8) and (2.11), the emissions are discrete and form clusters. Figure 3.2 summarises the sticky-HDP-HMM where $\lambda := (\mu_0, \kappa_0, \Psi_0, \nu_0)$.

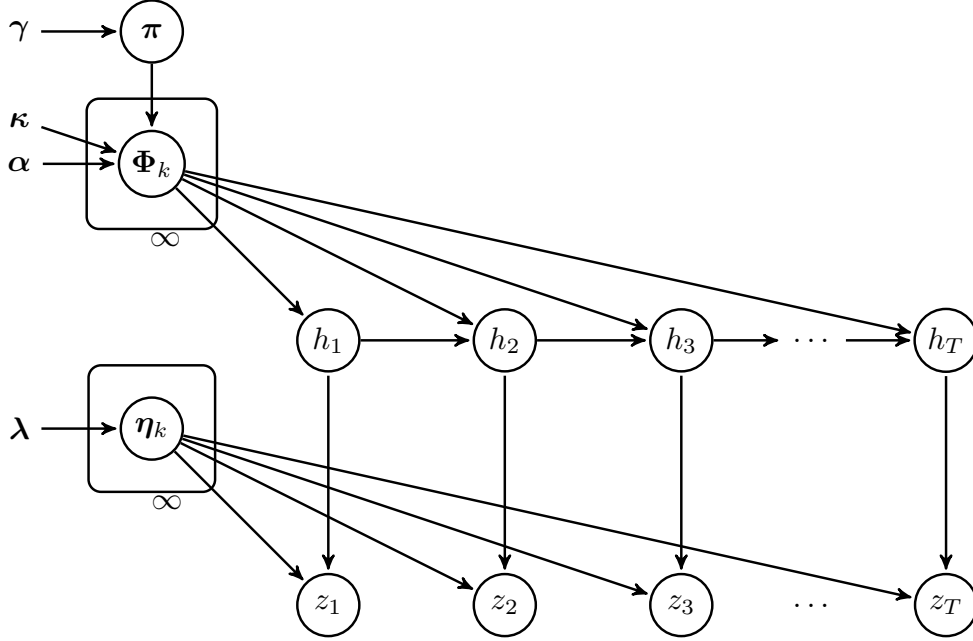


Figure 3.2: Sticky-HDP-HMM prior over the VAE latents. Global stick π couples every transition row; stickiness κ biases self-loops. Gaussian emissions share across hero classes.

Variational Inference. Given hyper-parameters $\mathcal{H} = \{\gamma, \alpha, \kappa, \mu_0, \kappa_0, \Psi_0, \nu_0\}$, we have joint density

$$p(z_{1:T}, h_{1:T}, \pi, \Phi, \eta \mid \mathcal{H}) = p(\pi) \prod_{k=1}^{\infty} p(\Phi_k \mid \pi) p(\eta_k) p(h_1 \mid \pi) \prod_{t=2}^T p(h_t \mid h_{t-1}, \Phi) \prod_{t=1}^T p(z_t \mid h_t, \eta). \quad (3.3)$$

We adopt similar methodology from [48] on variational inference with mean-field. Theoretically speaking there can exist infinite clusters with stick-breaking weight π_k such that $\sum_{k=1}^{\infty} \pi_k = 1$. For variational inference, we truncate the number of cluster to finite K and squeeze the remaining stick-mass part past current K to $(K+1)$ -th cluster. So we have

- $\pi_{1:K}$ - current top- K components,
- π_{K+1} - remainder of mass of infinitely many sticks beyond K ,
- $\sum_{k=1}^{K+1} \pi_k = 1$.

It is worth pointing out that the global GEM sticks π is the only non-conjugate part in the sticky HDP-HMM ELBO. As empirically explored in [49], π has small posterior variance in large datasets. We can impose a degenerate posterior on π for simplicity, i.e.

$$q(\pi) = \delta(\pi - \pi^*)$$

and optimise it numerically. However, since $p(\pi) \sim \text{GEM}(\gamma)$, it is not trivial to write out the distribution. Since we know the relationship of β and π , we can instead impose a degenerate posterior on β and construct π^* from β^* , i.e.

- $q(\beta) = \delta(\beta - \beta^*)$, we only need K parameters $\beta_1^*, \dots, \beta_K^* \in (0, 1)$.
- $\pi_k^* = \beta_k^* \prod_{l < k} (1 - \beta_l^*)$, $k = 1, \dots, K$,
- $\pi_{K+1}^* = 1 - \sum_{k=1}^K \pi_k^* = \prod_{k=1}^K (1 - \beta_k^*)$.

Therefore we have mean-field with amortisation,

$$q(z_{1:T}, h_{1:T}, \Phi, \eta) = \prod_{k=1}^K q(\Phi_k) q(\eta_k) \times q(h_{1:T}) \times \prod_{t=1}^T q_\phi(z_t \mid x_t, B_t, H).$$

- $q_\phi(z_t \mid x_t) = \mathcal{N}(\mu_t, \text{diag}(\sigma_t^2) + F_t F_t^\top)$ from encoder network.
- $q(h_{1:T})$ obtained by forward-backward HMM inference in (2.20). We denote $q(h_t = k) = \hat{r}_{tk}$, $q(h_t = j, h_{t+1} = k) = \hat{\xi}_{tjk}$.
- $q(\Phi_k) \sim \text{Dir}(\hat{\varphi}_{k1}, \dots, \hat{\varphi}_{k(K+1)})$.
- $q(\eta_k) \sim \text{NIW}(\hat{\mu}_k, \hat{\kappa}_k, \hat{\Psi}_k, \hat{\nu}_k)$.

For full derivation of updates on variational distribution and optimisation of π , see Appendix B.

β -KL Decomposition As mentioned in 3.1.2, in VAE training with standard normal prior $\mathcal{N}(0, I)$, we can decompose the KL divergence into mutual information, total correlation and per-dimension KL. However, in sticky HDP-HMM prior mode, the prior $p(z)$ is non-factorial. So the relationship $\text{KL} = \text{MI} + \text{TC} + \text{DW}$ identity does not hold. But total correlation can be calculated anyway since it is the structure of $q(z)$ only. In this case, we can still penalise total correlation heavily while keeping the rest of KL components moderately regularised.

3.3 On-Policy Skill-Aware PPO

We train an on-policy agent with proximal policy optimization (PPO) [20] that consumes (i) a VAE latent z_t and (ii) a causal belief over skills $\alpha_t \equiv q(h_t \mid z_{1:t})$ inferred online by the sticky HDP-HMM from Sec. 3.2. The PPO objective is shaped by an intrinsic reward that couples the world model and the skill process.

We use a recurrent actor-critic with a GRU core (hidden size $H = 256$). At each timestep, the policy inputs

$$x_t = [z_t; \alpha_t] \in \mathbb{R}^{D+K+1},$$

is embedded by a small MLP and then passed through the GRU: $[o_t; g_t] = \text{GRU}(f(x_t), g_{t-1})$. The actor produces masked logits $\pi_\psi(\cdot \mid o_t)$ over the MiniHack action subset available in the current environment, and 2 critic outputs $V_\psi^{\text{ext}}(o_t)$, $V_\psi^{\text{int}}(o_t)$ to track extrinsic and intrinsic (if apply) respectively. Hidden states are reset on episode termination.

In principle, one could substantially increase the representational power of the actor-critic (AC) network by stacking recurrent layers, adding larger MLP towers, or using advanced normalization and attention mechanisms. However, the central aim of this work is to study the impact of latent skill modelling (sticky HDP-HMM) and curiosity-driven intrinsic rewards on exploration and success rate in MiniHack. If the AC network is made very powerful, it risks outshining these contributions: a sufficiently expressive policy/value network may achieve high success rates even without structured skills or intrinsic bonuses, thereby masking the benefits of our proposed Bayesian skill space and curiosity signals.

To isolate the effect of our method, we therefore deliberately use a moderately expressive baseline AC: a single-layer GRU on top of a linear+ReLU feature encoder, with separate linear heads for policy and values. This architecture is strong enough to capture partial observability and temporal dependencies, but not so powerful that it can solve exploration-heavy tasks alone. By holding the AC capacity fixed and relatively simple, any observed performance gains can be attributed more directly to the skill-based latent structure and intrinsic reward shaping introduced in this thesis.

3.3.1 Causal skill inference for control

Hierarchical priors from offline posteriors For simple tasks, there could be only a few skill used. To prevent catastrophic forgetting of unused skills on simple tasks, we promote the offline posteriors to online priors at the start of PPO: for each skill k , the emission prior is set to the NIW posterior $(\mu_{0,k}, \kappa_{0,k}, \Psi_{0,k}, \nu_{0,k}) \leftarrow (\hat{\mu}_k, \hat{\kappa}_k, \hat{\Psi}_k, \hat{\nu}_k)$, and each transition row prior becomes the Dirichlet posterior base $\varphi_{0,k} \leftarrow \hat{\varphi}_k$ with tempered weight. The remainder state is excluded and is always kept as original prior. Streaming variational updates then proceed with these hierarchical priors.

Online emissions: Student- t point likelihood. At control time, the HMM conditions on a point embedding z_t^* — a reparameterised sample based on encoder mean μ_t and variance Σ_t from current observation screen, with NIW posterior on them. If we integrate out (μ_t, Σ_t) , we get a closed-form marginal likelihood for latent z_t that is multivariate Student- t distributed. Mathematically:

$$\begin{aligned} (\mu_k, \Sigma_k) &\sim \text{NIW}(\hat{\mu}_k, \hat{\kappa}_k, \hat{\Psi}_k, \hat{\nu}_0), \\ z_t^* \mid \mu_k, \Sigma_k &\sim \mathcal{N}(\mu_k, \Sigma_k), \\ \implies p(z_t^* \mid h_t=k) &= \text{St}_{\nu_k^*}\left(z_t^* \mid \hat{\mu}_k, \Sigma_k^*\right), \quad \nu_k^* = \hat{\nu}_k - D + 1, \quad \Sigma_k^* = \frac{\hat{\kappa}_k + 1}{\hat{\kappa}_k \nu_k^*} \hat{\Psi}_k, \end{aligned}$$

Thus, using a Student- t emission model is the Bayesianly consistent choice, as it accounts for parameter uncertainty in both the mean and covariance. Beyond theoretical correctness, the heavier tails of the Student- t distribution make the skill inference procedure more robust to outliers and latent representation drift, which are prevalent in online reinforcement learning. This ensures that the HMM’s posterior skill assignments remain stable even when the encoder distribution evolves during training.

Online transitions: VI-consistent log-potentials In the sticky HDP-HMM we maintain row-wise Dirichlet posteriors for the transition matrix:

$$q(\Phi_k) = \text{Dir}(\hat{\varphi}_{k1}, \dots, \hat{\varphi}_{k(K+1)}).$$

We use the expected log transition potentials under the Dirichlet rows derived in B.2,

$$\ell_{i \rightarrow j} = \mathbb{E}_{q(\Phi_k)}[\log \Phi_{ij}] = \psi(\hat{\varphi}_{ij}) - \psi\left(\sum_m^{K+1} \hat{\varphi}_{im}\right),$$

where $\psi(\cdot)$ is the digamma function. Intuitively, $\ell_{i \rightarrow j}$ encodes our current belief about the log-likelihood of transitioning from skill i to skill j . We can write the transition matrix used in filtering compactly as

$$\hat{A}_{ij} \propto \exp(\ell_{i \rightarrow j}) = \exp\left(\psi(\hat{\varphi}_{ij}) - \psi\left(\sum_m \hat{\varphi}_{im}\right)\right).$$

Causal filter: forward recursion for skill inference With emission likelihoods

$$B_{t,k} = p(z_t \mid h_t=k, \eta_k)$$

and log-potentials ℓ , the HMM filter recursions are

$$\alpha_1(k) \propto \pi_k B_{1,k}, \quad \alpha_{t+1}(k) \propto B_{t+1,k} \sum_i^{K+1} \exp(\ell_{i \rightarrow k}) \alpha_t(i). \quad (3.4)$$

Here

$$\alpha_t(k) = q(h_t = k \mid z_{1:t})$$

is the causal belief over skills at time t , i.e. the forward-filtered posterior given observations up to t . We also compute the one-step pair marginals:

$$\hat{\xi}_{t,jk} = q(h_t = j, h_{t+1} = k \mid z_{1:t+1}) \propto \alpha_t(j) \exp(\ell_{j \rightarrow k}) B_{t+1,k}.$$

These causal posteriors provide both policy features α_t and the skill-aware intrinsic terms below.

3.3.2 Intrinsic Reward: Model–Uncertainty and Skill Entropy

To accelerate exploration in sparse-reward MiniHack maps we augment the extrinsic environment reward r_t^{ext} with three information-theoretic bonuses:

$$r_t = r_t^{\text{ext}} + \eta_t^{\text{dyn}} r_t^{\text{dyn}} + \eta_t^{\text{bnd}} r_t^{\text{bnd}} + \eta_t^{\text{tr}} r_t^{\text{tr}}, \quad \eta_{\{\cdot\}} > 0.$$

(1) Dynamics-prediction surprise r_t^{dyn} We define a surprise term

$$S_t^{\text{dyn}} = D_{\text{KL}}[q_\phi(z_{t+1} \mid x_{t+1}) \parallel p_\theta(z_{t+1} \mid z_t, a_t, \alpha_t)],$$

where $q_\phi(z_{t+1} \mid x_{t+1})$ is the posterior over the next latent given the actual next observation, while $p_\theta(z_{t+1} \mid z_t, a_t, \alpha_t)$ is the predictive prior from the world model given the current latent, action and skill belief.

Intuitively, this measures how surprising the actual next latent z_{t+1} is compared to the agent’s dynamics model prediction. This KL is an instance of Bayesian surprise [50] at the latent level: it measures how much the posterior over z_{t+1} (after seeing x_{t+1}) diverges from the world model’s predictive prior. Under the variational world model approximation, this is equivalent to the information gain about the dynamics parameters θ .

$$\mathbb{E}_q \left[S_t^{\text{dyn}} \right] = I(\theta; z_{t+1} \mid z_t, a_t, h_t), \quad (3.5)$$

where $I(\cdot; \cdot)$ denotes mutual information. Thus the dynamics KL measures how much the model posterior over θ would change if this transition were observed.

This is directly related to active learning, where a learner actively selects actions that maximize expected information gain about the environment. In our setting, rewarding the agent with r_t^{dyn} encourages exploration toward transitions that are poorly predicted, i.e. those that are most informative for improving the world model. In this sense, dynamics KL acts as an intrinsic reward that aligns exploration with the principle of querying the environment to maximally reduce uncertainty, a central idea in active learning [30].

(2) Skill-boundary entropy (gated) r_t^{bnd} We define the skill entropy at time t from the filtered belief state α_t :

$$H_t = - \sum_k^{K+1} \alpha_{t,k} \log \alpha_{t,k}.$$

This entropy is high when the model is uncertain about the current skill, and low when one skill dominates. To focus the reward specifically on transitions between skills, we apply a causal gate:

$$g_t = \mathbb{I}\{H_t - H_{t-1} > \varepsilon\}, \quad B_t = H_t \cdot g_t.$$

We use the filtered entropies so the gate is causal. The threshold $\varepsilon > 0$ avoids spurious triggers from small fluctuations. This reward encourages the agent to explore contexts where the skill identity is ambiguous or changing, i.e. novel situations where the current cluster boundaries may be refined because skill entropy H_t is exactly the posterior uncertainty over the latent discrete state h_t and rewarding entropy increases corresponds to rewarding unexpected information gain about the skill process — a form of intrinsic motivation to discover new macro-states.

(3) Skill-transition novelty (off-diagonal) r_t^{tr} We have transition matrix used in filtering

$$\hat{A}_{jk} \propto \exp(\ell_{j \rightarrow k}).$$

Given the pairwise causal marginals $\hat{\xi}_{t-1,jk} = q(h_{t-1} = j, h_t = k \mid z_{1:t})$, we define

$$N_t = \sum_{j \neq k} \hat{\xi}_{t-1,jk} [-\log \hat{A}_{jk}],$$

The summation excludes diagonal terms ($j \neq k$), so we only reward cross-skill transitions. The factor $-\log \hat{A}_{jk}$ is the negative log-probability of the transition under the posterior Dirichlet expectation. It assigns higher reward to transitions that are rare under the current model, i.e. novel edges in the skill graph. Thus N_t is the expected surprisal of the transition event. Rewarding high surprisal aligns exploration with visiting edges that most update the posterior over transition dynamics.

Normalization and squashing. To stabilize scales without introducing negative shaping, we use scale-only normalization and bounded squashing:

- For unbounded signals (S_t^{dyn}, N_t), divide by a running exponential moving average standard deviation, then squash with $f(x) = 1 - \exp(-x/\tau)$, a monotone, concave, linear near zero function to map $[0, \infty) \rightarrow [0, 1]$.
- For the bounded entropy, normalize by its maximum $\log K_{\text{act}}$ before squashing.

$r_t^{\text{dyn}}, r_t^{\text{bnd}}, r_t^{\text{tr}} \in [0, 1]$ are the normalised-squashed terms. The coefficients are annealed $\eta_t = \eta_0 \exp(-t/\tau)$, so extrinsic reward dominates late-game fine-tuning.

3.3.3 PPO objective and recurrent update

We use the standard clipped PPO surrogate with GAE(λ) advantages $\hat{A}_t$ and clipped value loss,

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t \left[\min(r_t(\psi) \hat{A}_t, \text{clip}(r_t(\psi), 1-\epsilon, 1+\epsilon) \hat{A}_t) \right] - c_v \|V_\psi - \hat{R}_t\|_2^2 + c_{\text{ent}} \mathbb{E}_t[H(\pi_\psi(\cdot | o_t))], \quad (3.6)$$

where $r_t(\psi) = \exp(\log \pi_\psi(a_t | o_t) - \log \pi_{\psi_{\text{old}}}(a_t | o_t))$. We train with sequence mini-batches (a small set of environment trajectories); hidden states are reset at episode boundaries inside the batch. Action logits are masked to the env-specific MiniHack action subset.

3.4 Training Schedules

Our full pipeline for sequential skill preservation proceeds in four distinct stages: (i) pretraining the variational autoencoder (VAE) on NetHack human replay data, (ii) iterative refinement of HMM priors and VAE, and (iii) online reinforcement learning with PPO using both models. This staged design separates representation learning, skill discovery, and policy learning, while allowing Bayesian priors to be updated progressively.

3.4.1 Stage I: VAE Pretraining

We first pretrain a multimodal VAE using offline NetHack human replay data from the `nle.dataset`, prepared by [51]². Each batch consists of 32 games, each game containing a sequence of 32 screens. The latent prior is fixed to a standard Normal distribution $\mathcal{N}(0, I)$. To focus learning on the spatial map, we mask non-map regions of the screen.

²<https://github.com/facebookresearch/nle/blob/main/DATASET.md>

Algorithm 1 Stage I: Pretrain VAE

```
1: Initialise VAE parameters  $\theta, \phi$  with prior  $p(z) = \mathcal{N}(0, I)$ 
2: for batch = 1 to 100 do
3:   Sample 32 games  $\{\mathcal{D}^i\}_{i=1}^{32}$ , each with a sequence of 32 screens
4:   for each sequence  $x_{1:T}$  in batch do
5:     Mask non-map regions of  $x_{1:T}$ 
6:     Encode  $q_\phi(z_t|x_t)$  and decode  $p_\theta(x_t|z_t)$ 
7:     Compute ELBO with reconstruction + KL to  $\mathcal{N}(0, I)$ 
8:     Update  $\theta, \phi$  via gradient ascent
9:   end for
10: end for
```

Pretraining stabilises the latent space z_t so that the HMM later operates on meaningful embeddings. Masking non-map regions prevents spurious background features from dominating. The hyperparameters used are summarised in C.1. The latent space dimension is set as 96.

3.4.2 Stage II: Iterative Refinement of HMM–VAE

We iteratively refine the HMM and VAE in four alternating steps. For HMM step, we freeze the encoder and fit a sticky HDP-HMM on the saved latent sequences $\{z_t\}$. We use the entire dataset in one batch fit, 32 games with 3200 screens each, 2 of which completed within 3200 screens while the rest are truncated. The inference cycle updates emission potentials (via NIW posteriors), then performs HMM forward–backward to compute sufficient statistics, followed by posterior updates for NIW parameters and transition posteriors Φ . Optionally, we optimise the global stick-breaking weights π via β . For masked screens we set log-emission $\log B_t = 0$ and still update transitions normally; this preserves temporal continuity without injecting spurious emissions. In VAE step, the HMM posterior is blended into the VAE prior with an annealing weight to avoid early collapse and to progressively align the encoder with the skill space. This encourages the encoder to align its latent distribution with skill clusters. We repeat four iterations: in the first three, we perform only one HMM update step and do not optimise β . This prevents skill fragmentation/over-fitting while the encoder is still adapting; in the final iteration, we fit the HMM until its ELBO stabilises and also optimise β . This locks in a stable skill library and transition graph.

Algorithm 2 Stage II: Iterative Refinement

Require: Replay dataset; pretrained VAE (θ, ϕ) ; sticky HDP-HMM hyperparameters (α, κ, γ) ;

NIW prior $(\mu_0, \kappa_0, \Psi_0, \nu_0)$; anneal weights $\{\lambda_m\}_{m=1}^4$.

- 1: **for** $m = 1$ **to** 4 **do**
- 2: **Encode** all sequences with current VAE to obtain $\{z_{1:T}\}$ (store masks per frame)
- 3: **Fit HMM on** $\{z_{1:T}\}$ using Alg. 3 with:
 - $m < 4$: one update step, *no* β optimisation
 - $m = 4$: iterate to ELBO convergence, *with* β optimisation
- 4: **Set VAE prior** to blended form:

$$p_{\text{prior}}^{(m)}(z_t) = (1 - \lambda_m) \mathcal{N}(0, I) + \lambda_m \sum_k \hat{r}_{tk} \mathcal{N}(\mu_k, \Lambda_k^{-1})$$

where $(\pi_k, \mu_k, \Lambda_k)$ are from the fitted HMM posteriors

- 5: **Retrain VAE** on replay data under $p_{\text{prior}}^{(m)}(z_t)$ (ELBO with all decoder heads)
 - 6: **end for**
-

Algorithm 3 Sticky HDP-HMM fit on $\{z_t\}$ (one step or to convergence)

Require: Latent sequences $\{z_{1:T}\}$; masks $\{m_t\}$; current posteriors (NIW, Dirichlet rows, β)

- 1: **repeat**
 - 2: **Emissions:** form log-likelihoods $\log B_{t,k} = \log p(z_t | h_t = k, \eta_k)$; if $m_t=0$, set $\log B_{t,k}=0$ for all k
 - 3: **Transitions:** expected log-potentials $\ell_{i \rightarrow j} = \psi(\hat{\phi}_{ij}) - \psi(\sum_m \hat{\phi}_{im})$
 - 4: **Forward-Backward:** compute $\hat{r}_{t,k}, \hat{\xi}_{t,jk}$ (respecting masks)
 - 5: **NIW update:** update $(\hat{\mu}_k, \hat{\kappa}_k, \hat{\Psi}_k, \hat{\nu}_k)$ from soft moments of $\{z_t\}$
 - 6: **Row Dirichlet update:** $\hat{\phi}_{k\cdot} \leftarrow \alpha\pi + \kappa\delta_k + \sum_t \hat{\xi}_{t,k}$.
 - 7: **if** “with β optimisation” **then**
 - 8: **Optimise global sticks:** $u_\beta \leftarrow \arg \min \mathcal{L}_\beta$ (ELBO-restricted, early-stopped)
 - 9: Recompute $\pi \leftarrow \text{SB}(\sigma(u_\beta))$ and refresh $\ell_{i \rightarrow j}$
 - 10: **end if**
 - 11: **until** ($m < 4$ and one pass done) **or** ($m = 4$ and ELBO converged)
-

The four-iteration loop implements a gentle posterior $\rightarrow$ prior coupling: early iterations keep the HMM conservative (one update, no global reweighting), so the encoder can realign to the emerging clusters without collapsing. The final iteration locks in the skill library by fully refining emissions, transitions, and the global sticks. This staged annealing yields stable skills and a latent space aligned with long-horizon structure, which improves credit assignment and downstream PPO performance. The hyperparameters used in this stage are summarised in C.2.

3.4.3 Stage III: Online PPO with VAE + HMM

Finally, we train a downstream policy using PPO on MiniHack environments. The policy conditions on both z_t and filtered beliefs α_t from the HMM. We periodically refresh the VAE and HMM posteriors with new replay buffers. This couples representation, skill space, and control in

a continual Bayesian loop.

Two mechanisms are critical for stable online HMM inference:

- **Tempered posterior-to-prior updates.** At the beginning of online training, we set posterior parameters as priors with temper weight (here 0.2) applied on statistics. This preserves previously discovered skills while allowing incorporation of new evidence, preventing catastrophic forgetting.
- **Streaming updates.** We update emission posteriors with an exponential moving average of sufficient statistics, controlled by ρ_{emission} (and similarly $\rho_{\text{transition}}$). This prevents abrupt parameter shifts while allowing adaptation to new observations.

Algorithm 4 Stage III: Online PPO with Skill Space

```

1: Initialise PPO policy  $\pi_\psi(a_t|z_t, \alpha_t)$  and value functions
2: Set Sticky HDP-HMM posterior as prior with tempered weight (0.2) on statistics
3: for training steps  $t = 1$  to  $T$  do
4:   Collect rollouts with policy  $\pi_\psi$ 
5:   Encode observations with VAE to obtain  $z_t$ 
6:   Run HMM filter to compute skill belief  $\alpha_t$ 
7:   Compute extrinsic reward  $r_t^{\text{ext}}$  and intrinsic bonuses  $r_t^{\text{dyn}}, r_t^{\text{bnd}}, r_t^{\text{tr}}$ 
8:   Update PPO policy using advantages from  $r_t^{\text{ext}} + \eta_t \cdot r_t^{\text{int}}$ 
9:   if periodic update step then
10:     Update sticky HDP-HMM posteriors with replay buffer latents:
       • Update emission NIW posteriors with streaming blend ( $\rho_{\text{emission}}$ ).
       • Update transition Dirichlet posteriors with  $\rho_{\text{transition}}$  (default  $\rho_{\text{transition}} = \rho_{\text{emission}}$ ).
11:     Update VAE with replay buffer spans
12:   end if
13: end for

```

This stage unifies representation, skills, and policy learning. Streaming updates (ρ) enable gradual posterior adaptation without destabilising old skills, while tempered posterior statistics set for prior preserves knowledge across tasks. Together, these mechanisms provide a Bayesian continual learning loop that avoids skill collapse. PPO leverages both z_t and α_t for robust control in sparse-reward MiniHack maps. The hyperparameters used in this stage are summarised in C.3.

Chapter 4

Results and Analysis

In this chapter we will present tests results and analysis on pretraining of VAE, VAE + HMM finetune and online PPO training.

4.1 VAE Pretraining

As mentioned in 3.4.1, we used human-played NetHack data to pretrain VAE to capture the representation of the game. One fragility of VAE is prone to posterior collapse. We have performed the following latent analysis to show our VAE training does not have posterior collapse.

4.1.1 Effect of the prior on latent geometry

We compare the latent statistics of the VAE trained with a standard Normal prior $p_0(z) = \mathcal{N}(0, I)$ (Fig. 4.1) against the VAE trained with a sticky HDP-HMM prior

$$p_{\text{HMM}}(z) = \sum_{k=1}^K \pi_k \mathcal{N}(z; \mu_k, \Lambda_k^{-1}),$$

whose emission parameters carry an NIW posterior (Fig. 4.2). For both models we report the aggregated posterior moments used throughout the analysis:

$$\Sigma_{\text{agg}} = \underbrace{\text{Var}_x[\mu_x]}_{\text{between-sample}} + \underbrace{\mathbb{E}_x[\text{diag}(\sigma_x^2) + F_x F_x^\top]}_{\text{within-sample}}, \quad \bar{\mu} = \mathbb{E}_x[\mu_x].$$

The two panels in each figure plot the component-wise mean $\bar{\mu}_i$ and the componentwise variance $[\Sigma_{\text{agg}}]_{ii}$.

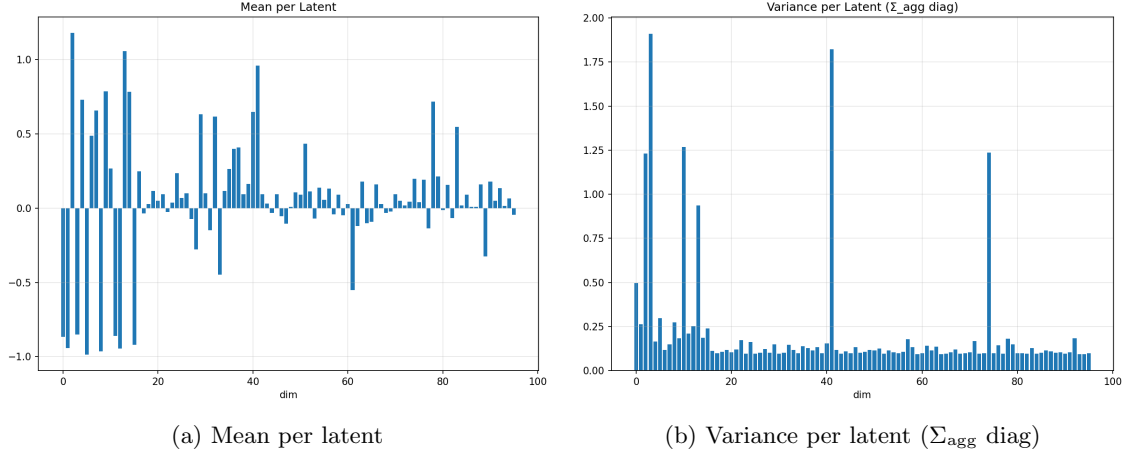


Figure 4.1: Plain VAE posterior ($p_0(z) = \mathcal{N}(0, I)$). A few axes carry most of the variation (large spikes), while many coordinates remain near the prior (small variance, mean near 0).

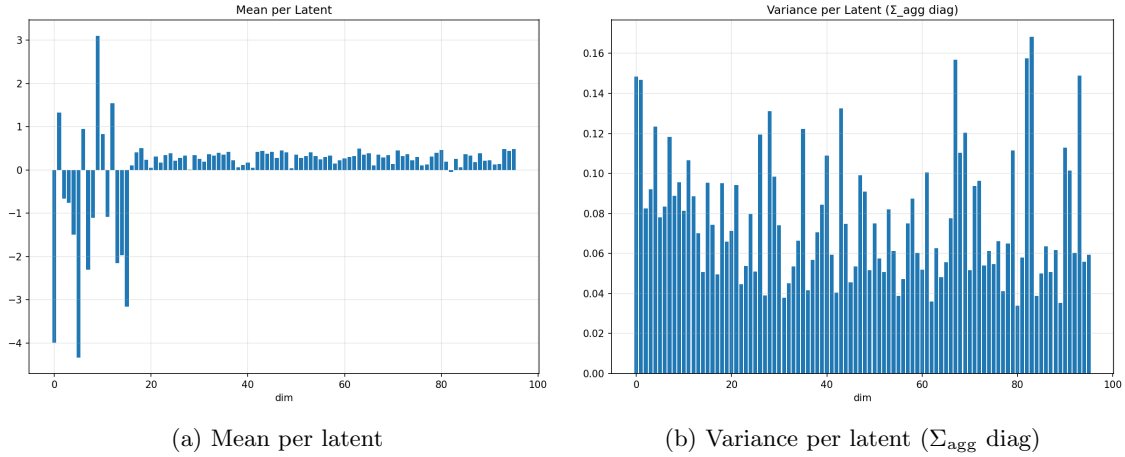


Figure 4.2: HMM-prior VAE posterior ($p_{\text{HMM}}(z)$). Variances are shrunk and homogeneous; several coordinates show structured, nonzero offsets that reflect the learned skill means μ_k . The mixture prior breaks rotational symmetry and yields a compact, skill-aligned latent space.

Plain Normal prior In Fig. 4.1 we observe:

- **Means close to zero for most coordinates.** Most $\bar{\mu}_i$ are near 0, with occasional larger excursions on the same axes that show high variance. Again this indicates that only a few coordinates carry substantial signal, while many dimensions behave like the prior.
- **Axis spikes in variance.** A small subset of coordinates exhibits very large aggregated variances ($\gtrsim 1$ –2), while many coordinates lie near a low baseline (≈ 0.05 –0.15). This “few-active-axes” pattern is typical of VAEs under an isotropic prior: the model concentrates information into a handful of directions and leaves many dimensions close to the prior,

Together these features reflect the rotational/scale symmetry of $p_0(z) = \mathcal{N}(0, I)$: the reconstruction pressure can freely rotate and stretch a few axes, producing anisotropy and leaving many dimensions weakly used.

Sticky HDP–HMM prior In Fig. 4.2 we see the complementary behaviour:

- **Structured mean offsets.** Several coordinates exhibit systematic nonzero means: these reflect the learned skill centres μ_k and the encoder’s alignment to them (the active skills trained from data are around 15). Unlike the standard prior, where offsets appear only on a few “hot” axes, the HMM prior induces piecewise-stationary offsets that encode macro-context information.
- **Shrunk and more homogeneous variances.** The aggregated per-coordinate variances are small and lie in a narrow band (here $\lesssim 0.15\text{--}0.17$) without extreme spikes. This is consistent with the NIW emission posterior: within-skill covariances are shrunk toward the HMM precision $E[\Lambda_k]$, and because z_t is pulled toward μ_{h_t} , the between-sample spread $\text{Var}_x[\mu_x]$ remains controlled. The result is a compact, well-conditioned latent geometry rather than a few blown-up axes.

The mixture prior thus breaks the rotational symmetry of the standard Normal: each skill contributes a mode in latent space, and the encoder is regularised to place z_t near the corresponding μ_{h_t} with skill-specific precision. This prevents axis blow-ups and distributes signal more evenly across coordinates in a way that is consistent with the sequential structure.

The Normal prior encourages a small set of highly active axes while leaving many dimensions close to the prior; this is efficient for reconstruction but risks partial collapse and unstable scales (large spikes). The sticky HDP-HMM prior, by contrast, acts as an empirical Bayesian codebook of skills: z_t is attracted to state-specific means with controlled precision, yielding tight, interpretable clusters and a stable scale across dimensions. This skill-aligned latent geometry is precisely what the HMM filter and the skill-based intrinsic rewards exploit in downstream control.

4.1.2 KL diagnostics under different priors

Figures 4.3 and 4.4 compare the latent statistics when the VAE is trained with a standard Normal prior $p_0(z) = \mathcal{N}(0, I)$ versus a sticky HDP-HMM mixture prior $p_{\text{HMM}}(z) = \sum_k \pi_k \mathcal{N}(z; \mu_k, \Lambda_k^{-1})$.

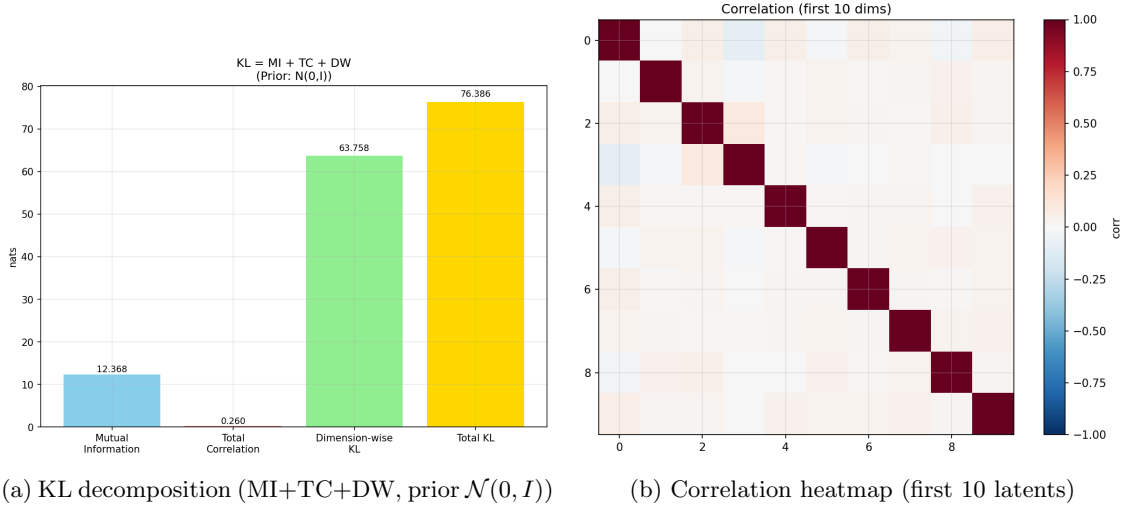


Figure 4.3: Plain-prior VAE. Large DW and near-diagonal correlation indicate an axis-aligned, almost independent representation.

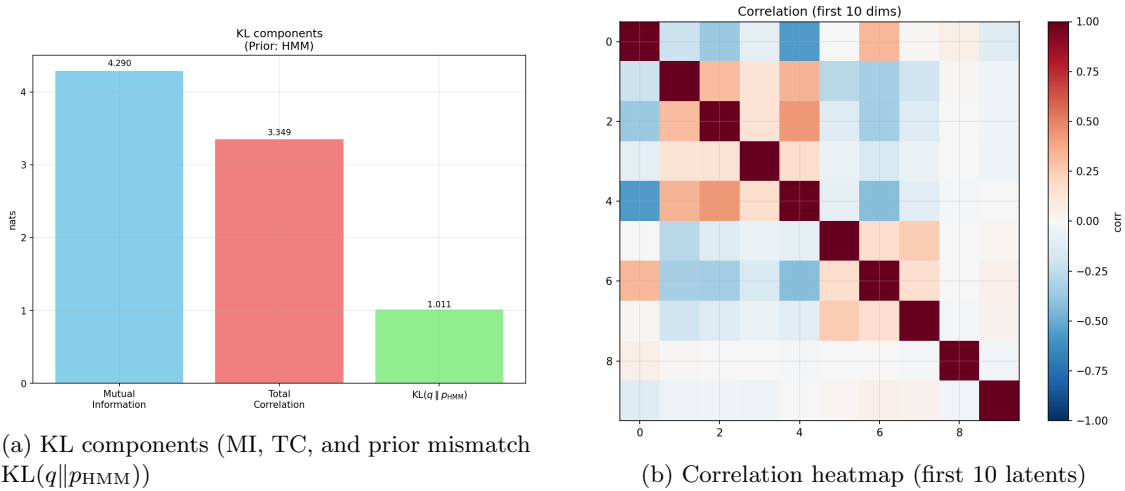


Figure 4.4: HMM-prior VAE. Stronger off-diagonal correlations (larger TC) and a small global mismatch to the prior indicate a skill-aligned, correlated representation well matched by the mixture prior.

For the standard Normal prior (Fig. 4.3a) we use the β -TCVAE identity which is valid because p_0 factorises across dimensions. In the HMM case (Fig. 4.4a) the prior is non-factorial; we therefore report three non-additive components: (i) a prior-agnostic MI (Gaussian aggregate approximation), (ii) TC of the aggregated $q(z)$, and (iii) the prior mismatch $KL(q(z) || p_{HMM}(z))$ (estimated by Monte-Carlo). The right panels (b) in both figures show the correlation matrix of the first ten latent coordinates of the aggregated posterior $q(z)$.

Standard Normal prior We observed two features:

- **Large DW, tiny TC.** The DW term dominates the total KL, while TC is nearly zero. This indicates that $q(z)$ is approximately factorised, but many marginals q_j deviate from

$\mathcal{N}(0, 1)$ (non-unit variance or non-zero mean). The correlation heatmap (Fig. 4.3b) confirms near-diagonality with very small off-diagonals.

- **Moderate MI.** MI is clearly positive (latent depends on x), but most of the mismatch to the prior is carried by per-coordinate shifts/scales (DW), consistent with the mean/variance spikes observed elsewhere in the plain-prior model. Overall, the isotropic prior pushes the encoder towards an axis-aligned and almost independent representation.

HDP-HMM prior With the mixture prior the picture changes qualitatively:

- **Non-negligible TC.** Even though we have applied penalty on TC, it is still sizeable, reflecting that the aggregated $q(z)$ retains meaningful inter-dimensional correlations. The correlation heatmap (Fig. 4.4b) shows pronounced off-diagonal structure (both positive and negative), in stark contrast to the plain-prior case. This is expected: each HMM emission has its own precision $E[\Lambda_k]$ and the latent space aligns to those skill-specific covariances.
- **Small prior mismatch.** The measured $\text{KL}(q(z) \parallel p_{\text{HMM}}(z))$ is close to zero (here ≈ 1 nat), showing that the HMM prior absorbs much of the aggregate structure in $q(z)$ (both means μ_k and within-skill covariances), so the joint fit between q and the prior is tight.
- **Reduced MI.** Relative to the plain prior, MI is lower. Intuitively, the HMM prior provides a codebook of skill means $\{\mu_k\}$ and precisions $E[\Lambda_k]$; the encoder is regularised to map inputs to these skill neighbourhoods, which compresses idiosyncratic variation and reduces the amount of information z must carry about x .

The standard Normal prior encourages an axis-wise, almost independent latent (small TC, large DW), leaving the prior far from the aggregated posterior along many coordinates. The sticky HDP-HMM prior instead breaks the rotational symmetry: it models clustered and correlated structure through skill-specific Gaussians, yielding (i) stronger inter-dimensional structure in $q(z)$ (larger TC), (ii) a very small global mismatch between q and the prior, and (iii) a moderate MI because much structure is captured by the prior itself. This skill-aligned geometry is precisely what the HMM filter and skill-based intrinsic rewards exploit downstream.

4.1.3 PCA/t-SNE comparison under different priors

Figures 4.5 and 4.6 compare the latent geometry obtained with a standard Normal prior ($p_0(z) = \mathcal{N}(0, I)$) and with a sticky HDP-HMM mixture prior ($p_{\text{HMM}}(z) = \sum_k \pi_k \mathcal{N}(z; \mu_k, \Lambda_k^{-1})$). In both figures the left panel shows a PCA projection of the posterior means μ_x onto the first two principal components; the right panel shows a t-SNE embedding.

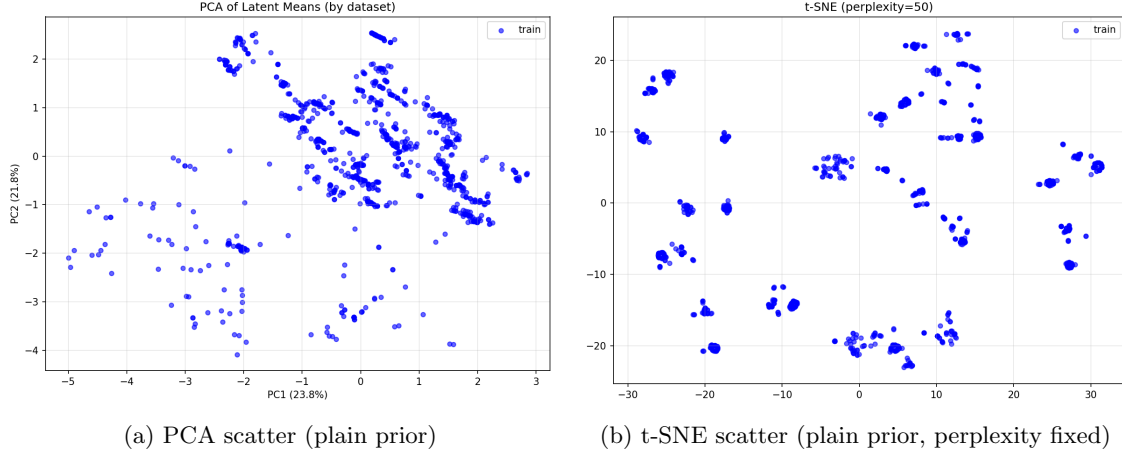


Figure 4.5: Plain-prior VAE: modest linear separability ($PC1 \approx 24\%$) and many micro-islands in t-SNE that are not tied to semantic skills.

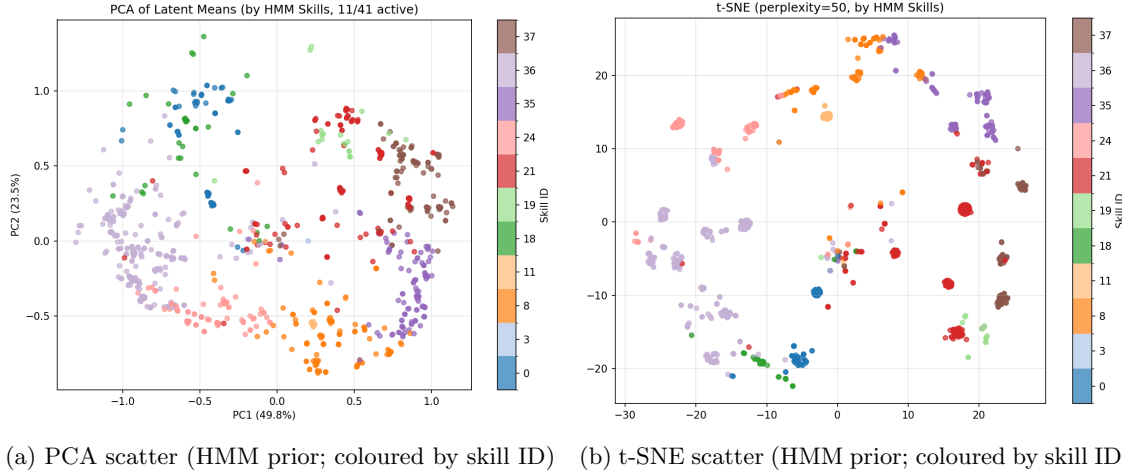


Figure 4.6: HMM-prior VAE: crisp, skill-aligned clusters. $PC1$ captures $\approx 50\%$ of the variance; t-SNE islands correspond to individual skills with small within-island spread and large between-island gaps.

PCA Comparison:

- **Standard prior** The scatter forms a broad cloud with modest explained variance on $PC1$ (here $\approx 24\%$). Multiple small local groupings are visible but there is no clear global mode structure. This is consistent with the isotropic prior: the encoder is free to rotate/scale a few axes and the between-sample variability is not anchored to discrete centres.
- **HMM prior** The first two PCs now capture a much larger share of variance (here $PC1 \approx 50\%$, $PC2 \approx 13\%$), and the projection shows crisp clusters. When coloured by posterior skill assignments, clusters align one-to-one with skill IDs. The increased linear separability indicates that most dispersion is explained by between-skill means $\{\mu_k\}$ imposed by the mixture prior.

t-SNE Comparison:

- **Standard prior** The embedding contains many micro-islands of points. Because there is no discrete prior structure, these islands mostly reflect local neighbourhoods created by the optimisation dynamics and temporal proximity in the data; they are not anchored to meaningful macro-states.
- **HMM prior** Islands become well-separated and each island concentrates points from a single skill (colour). Within-island spread is small, reflecting the NIW-shrunk within-skill covariance; between-island distances are large, reflecting the differences between μ_k . The embedding is thus mode-structured: clusters are interpretable skills rather than incidental neighbourhoods.

The standard Normal prior yields a latent space with weak global structure: PC1 explains little variance, clusters are not tied to semantic skills, and t-SNE islands are numerous but unanchored. The sticky HDP-HMM prior breaks the rotational symmetry of $\mathcal{N}(0, I)$ by introducing piecewise-stationary Gaussian components; the encoder is regularised to map observations near the corresponding skill means with skill-specific precision. As a result, (i) PCA shows strong between-skill separation, (ii) t-SNE reveals distinct, compact islands aligned with skills, and (iii) the latent geometry becomes skill-aligned and interpretable, exactly the structure exploited by the HMM filter and the skill-based intrinsic bonuses in downstream control.

4.2 Sticky HDP-HMM Fitting

We summarize the status of the HMM after the variational E-step using five complementary plots (Fig. 4.7). All figures are produced by (i) encoding batches with the VAE to obtain $\{(\mu_t, \Sigma_t)\}_t$, (ii) forming expected emission log-likelihoods $B_{t,k}$ under the current NIW posteriors, (iii) using the HMM’s current transition posteriors to obtain expectations $E[A]$, and (iv) aggregates occupancies, dwell statistics and Viterbi paths for visualisation.

(a) Transition matrix heatmap We plot the row-wise expectations $E[A_{ij}]$ of the transition matrix (Fig. 4.7a). The diagonal mass reflects stickiness: larger $E[A_{kk}]$ implies longer expected dwell times in state k ($\mathbb{E}[L_k] \approx 1/(1 - E[A_{kk}])$ for a geometric dwell model). The reported diagonal mean ($\overline{E[A_{kk}]} = 0.391$ in this run) is an aggregate across all states, including low-occupancy ones that remain near the sticky prior, hence it should be read together with the dwell-time panel below. Off-diagonal structure highlights which state-to-state transitions the model uses; faint but systematic bands indicate preferred “routes” between skills.

(b) Dwell-time PMFs (empirical vs. geometric) For the top five occupied states we compare the empirical histogram of dwell lengths with the geometric PMF implied by the estimated self-transition probability $p_{\text{stay}} = E[A_{kk}]$ (red curve) (Fig. 4.7b). A good fit (histogram close to the geometric curve) indicates that the sticky Markov-chain assumption is adequate for those skills; heavier empirical tails suggest either (i) lingering within heterogeneous sub-modes that the

current K has not separated, or (ii) segmentation noise from the encoder. The panel also annotates each state’s marginal occupancy $\hat{\pi}_k$ (from the forward–backward posteriors), which helps interpret which skills dominate the corpus.

(c) Emission means in 2-D We embed the skill means $\{\mu_k\}$ (size $\propto \hat{\pi}_k$) using t-SNE (Fig. 4.7c). Large separations confirm that the emission components define distinct macro-contexts in latent space. When two means appear near each other, their covariances (not shown in t-SNE) or transition connectivity typically differentiate them.

(d) Skill occupancy (effective K) We plot the fitted occupancies $\hat{\pi}_k$ and the effective number of active skills $\text{eff} K$ derived from them (Fig. 4.7d). Concentration on a small subset (here $\text{eff} K \approx 9$) indicates that the data uses only a few skills heavily, while the remainder stay near the prior and are rarely visited. This is the expected behaviour under a hierarchical DP prior: the tail states carry small mass until new evidence promotes them.

(e) Skill rasters (Viterbi paths) We decode Viterbi paths on several long sequences and display the state label per frame (Fig. 4.7e). Horizontal runs show dwell within a skill; sharp colour changes mark boundaries. Long, coherent runs indicate that the model has found persistent macro-actions; frequent short runs suggest either genuinely rapid switching tasks or over-segmentation. Because the raster is decoded with the current $E[\log A]$ and emissions, it is a direct sanity-check that the learned transitions and emissions produce plausible causal segmentations.

The five diagnostics are designed to agree: (i) blocks on the diagonal of $E[A]$ and large p_{stay} imply long runs in the raster and slowly decaying geometric curves in the dwell panel; (ii) large $\hat{\pi}_k$ bars correspond to large markers in the t-SNE of μ_k ; Finally, the diagonal mean of $E[A]$ should be read together with the state-wise dwell fits: global averages can be small even when the active states are highly persistent.

4.3 PPO Ablations

The baseline model in our work is VAE + Sticky HDP-HMM skill space + 3 types of curiosity-driven rewards. We will perform ablation runs to evaluate the performance of each module. In addition, we have exploration via random network distillation (RND) [22] as a benchmark of previous work.

RND provides a simple, task-agnostic novelty signal. Let x_t be the observation (or its embedding $\phi(x_t)$). RND fixes a random target network $f : \phi(x) \mapsto \mathbb{R}^m$ and trains a predictor network g_ψ online to regress onto f on the states the agent visits. The intrinsic reward is the prediction error

$$r_t^{\text{rnd}} = \|g_{\psi^*}(\phi(x_t)) - f(\phi(x_t))\|_2^2, \quad \psi^* = \min_{\psi} \mathbb{E}[\|g_{\psi}(\phi(x_t)) - f(\phi(x_t))\|_2^2].$$

Unvisited (or rarely visited) states yield large errors and hence large bonuses; as the predictor learns those states, the error decreases, implementing a count-like exploration signal. In practice

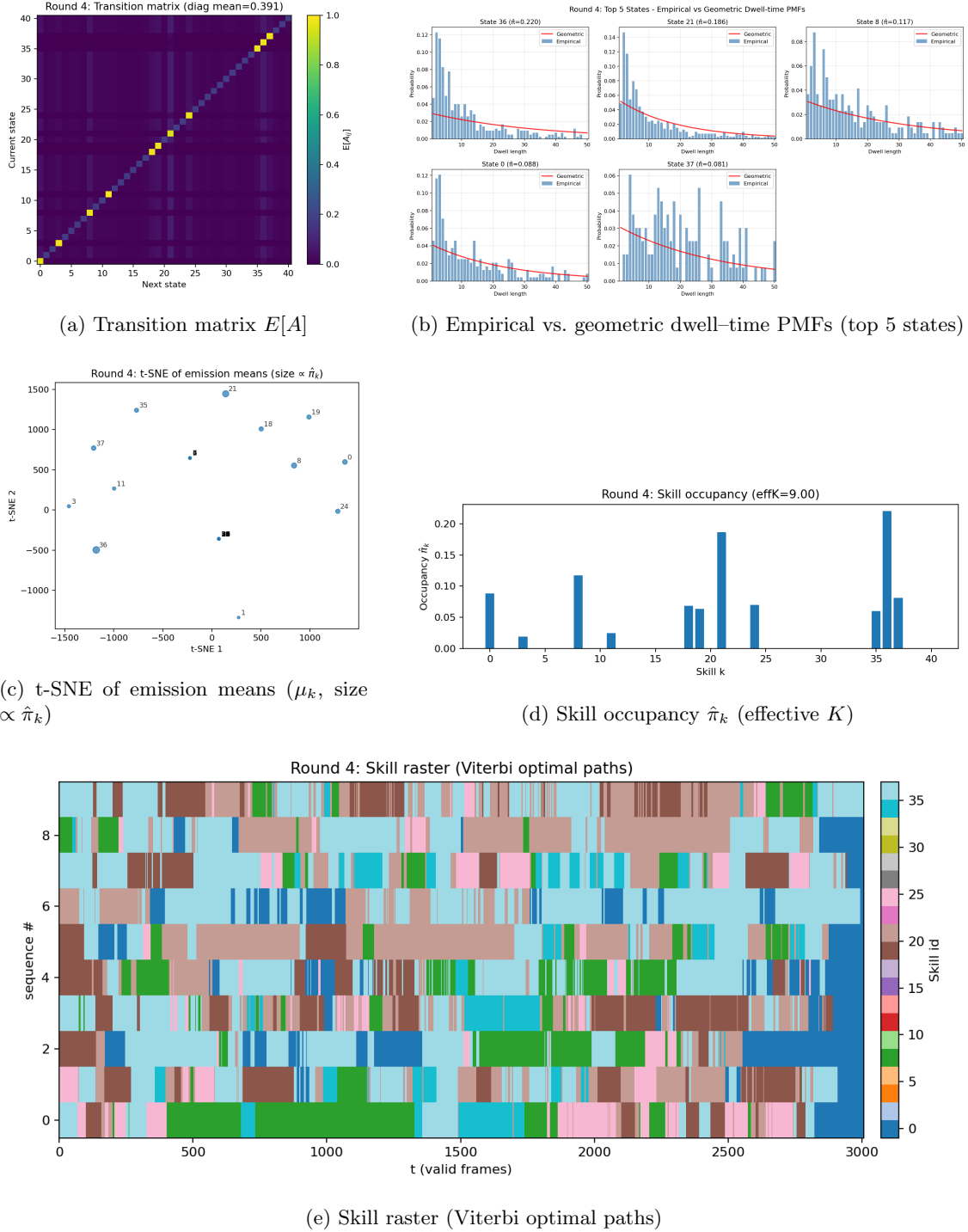


Figure 4.7: Sticky HDP-HMM diagnostics after the variational E-step. Panels are generated by our analysis utilities and summarise transitions, dwell statistics, emission geometry, occupancies and causal segmentations.

we normalise and squash r_t^{rnd} (EMA standard deviation; $1 - e^{-x/\tau}$) and mix it with extrinsic reward under PPO in exactly the same way as our other bonuses.

RND is a strong, widely adopted baseline for sparse-reward exploration (Atari [52], DM Control [53, 54], MiniGrid [55, 54]), requiring no task priors and integrating cleanly with on-policy and off-policy RL. Its intrinsic signal is stationary as the target network is fixed, so it cleanly isolates the benefit of “novelty seeking” without confounding representation updates from a world model. Additionally, because RND does not impose temporal abstraction or a skill library, it serves as a conservative comparator for our method: improvements over RND can be attributed to the Bayesian skill space (sticky HDP-HMM) and the skill-aware bonuses, rather than to generic novelty alone. We use the same PPO backbone and the same normalisation / annealing schedule to ensure an apples-to-apples comparison in our ablations.

We report (i) success rate (successfully complete the goal within maximum episode steps), (ii) undiscounted extrinsic return (sum of environment rewards) which contains both success and truncated episodes and (iii) episode length.

4.3.1 MiniHack Room 15×15 : ablations and analysis

We start our test with simplest environment. There are no traps, monsters or teleports. The goal for the agent is to reach `>`. The available actions are navigation keys: $[N, E, S, W, NE, SE, SW, NW]$. Fig 4.8 demonstrates an example of the game.

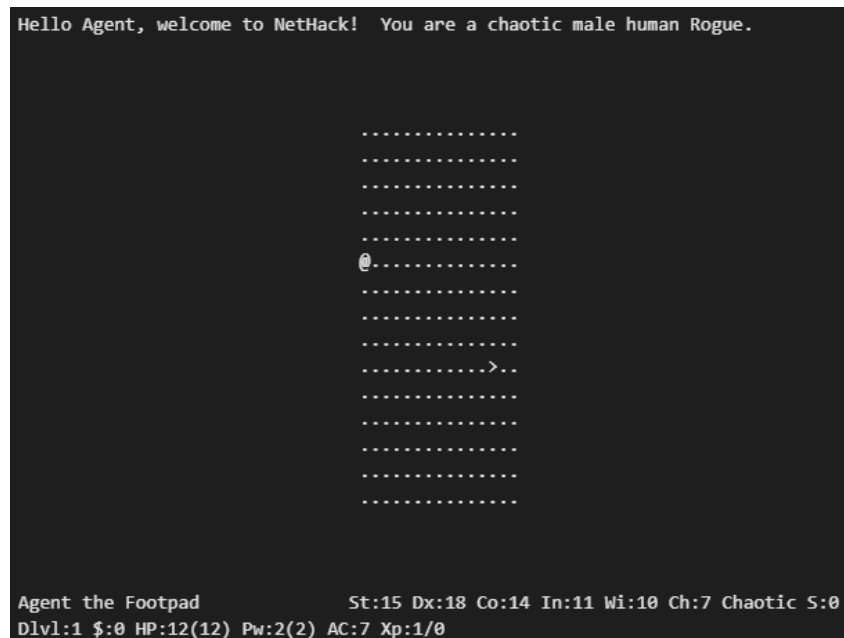


Figure 4.8: MiniHack Room 15×15

We train PPO for 2×10^5 steps starting from a freshly initialised actor-critic and a pretrained VAE (with/without the sticky HDP-HMM prior). We evaluate nine configurations (Tab. 4.1): no-HMM baselines, HMM alone, and HMM with each intrinsic component (dynamics surprise, skill-boundary entropy, transition novelty), as well as the full curiosity suite and an RND baseline.

MiniHack-Room-Random-15x15-v0	Success Rate (%)	Episode Extrinsic Return	Episode Length
No HMM, No intrinsic	37.08	0.243 ± 0.548	230.2 ± 103.8
No HMM, dynamic surprise only	41.78	0.258 ± 0.583	223.2 ± 106.0
No HMM, RND	41.94	0.259 ± 0.576	220.5 ± 108.4
HMM, No intrinsic	42.99	0.168 ± 0.618	226.1 ± 102.7
HMM, dynamic surprise only	45.10	0.190 ± 0.630	221.3 ± 105.4
HMM, skill entropy only	40.05	0.133 ± 0.622	230.1 ± 101.9
HMM, transition novelty only	42.52	0.174 ± 0.626	223.2 ± 106.2
HMM, full curiosity	45.30	0.195 ± 0.630	217.8 ± 106.9
HMM, RND	43.20	0.165 ± 0.645	223.3 ± 105.5

Table 4.1: MiniHack Room Random 15x15 Statistics

The main outcome is success rate; we also report episode extrinsic return (undiscounted sum of environment rewards) and the episode length. We plotted the success rate comparison for different ablation tests in Fig 4.9 and intrinsic reward decompositions in Fig 4.10. The success rate is cumulative. At the beginning of the training, the completed episodes are few so the success rate could be volatile.

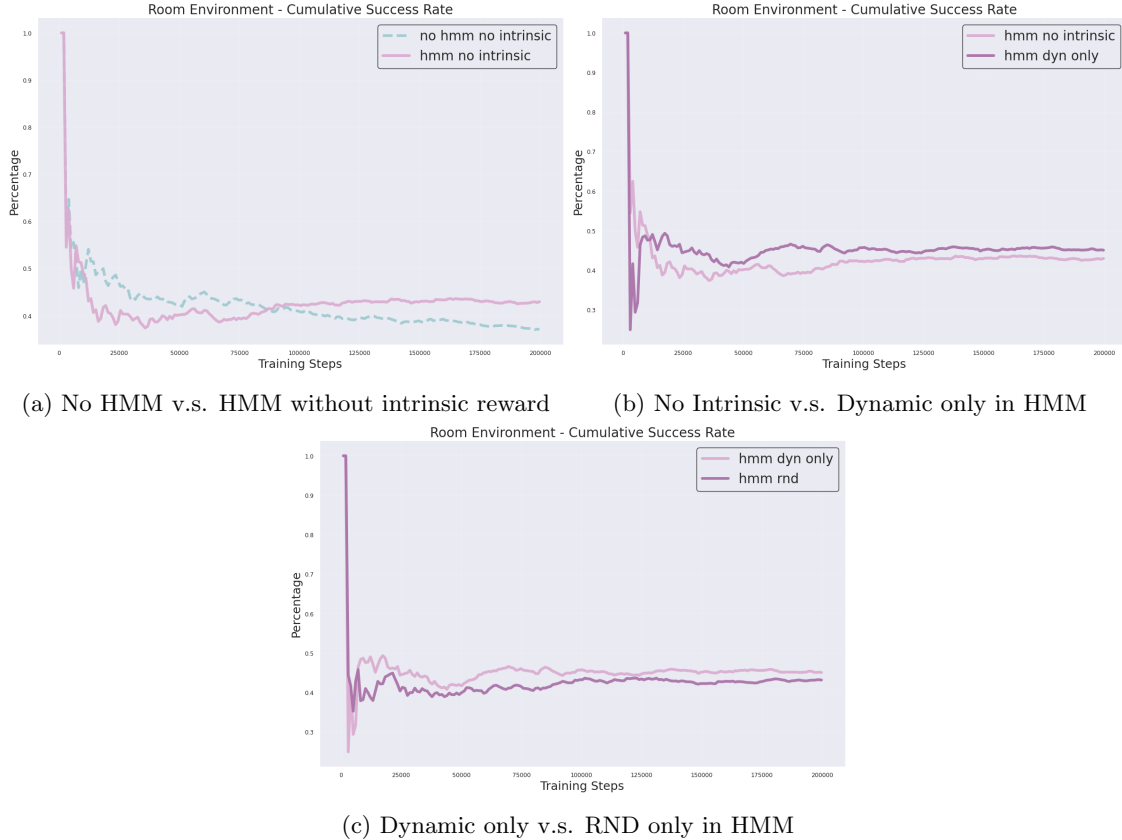


Figure 4.9: Ablation Study of effects of HMM and Intrinsic Reward in MiniHack Room.

We have following observation and analysis:

- **HMM improves over no-HMM.** Without any intrinsic bonuses the HMM prior increases success from 37.1% to 43.0% (+5.9 points). Intuitively, the sticky HDP-HMM imposes

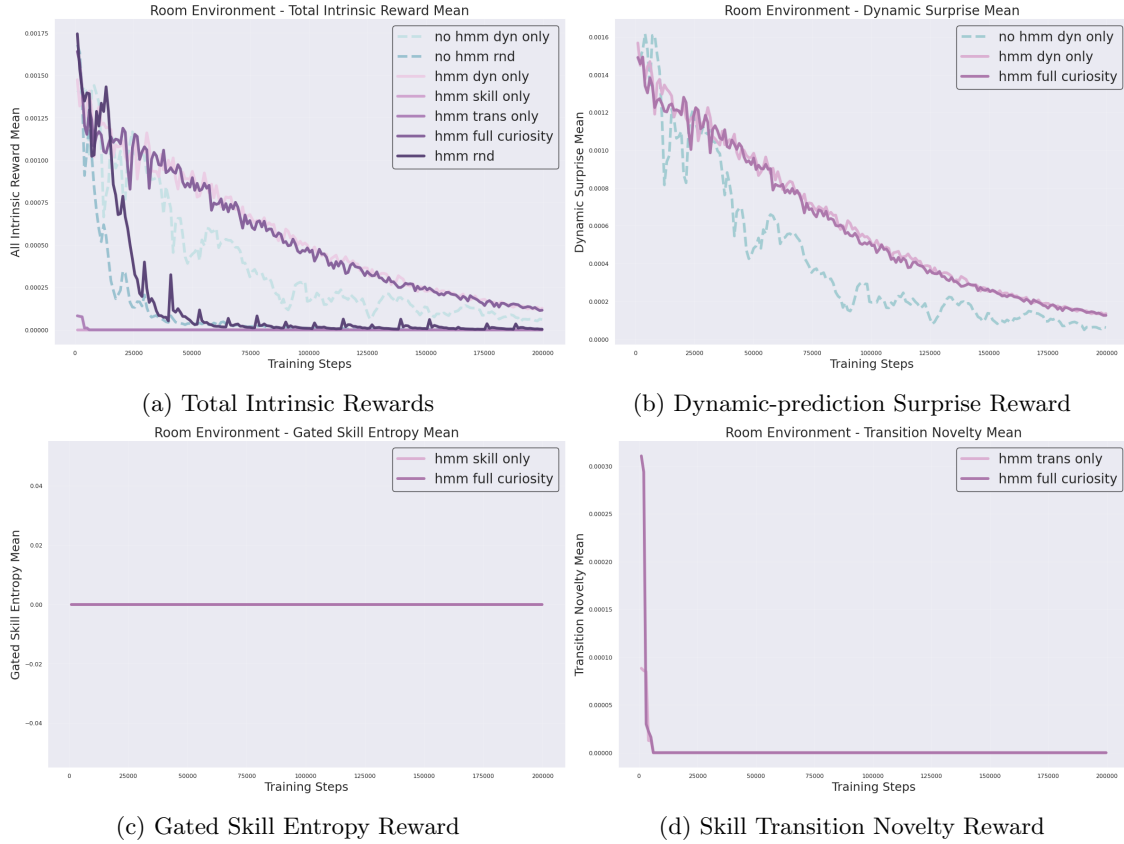


Figure 4.10: Intrinsic Reward Means in MiniHack Room.

piecewise-stationary structure on z_t , encourages persistent skills, and reduces dithering in the partially observed map; the filtered belief α_t supplies a stabilised, slowly varying signal to the policy. This helps even in a simple room.

- **Dynamics surprise helps early exploration.** Adding only the dynamics-prediction bonus lifts success further (no-HMM: 37.1% $\rightarrow$ 41.8%; HMM: 43.0% $\rightarrow$ 45.1%). The bonus is largest early (Fig. 4.10b) and decays as the world model becomes confident, which is exactly the desired behaviour in a small map with sparse extrinsic reward.
- **Full curiosity yields the best success and the shortest episodes.** The HMM + full-curiosity configuration attains the highest success (**45.3%**) and the shortest episodes (**217.8 $\pm$ 106.9** steps). In this environment the extra curiosity terms (skill entropy, transition novelty) are almost always gated off (Fig. 4.10c-d); thus the gain over “HMM, dyn only” is small.
- **Full curiosity does not yields highest extrinsic return.** The discrepancy is not due to longer trajectories (indeed, HMM+full has the shortest episodes), but to how extrinsic reward is accumulated in this room. Two effects dominate: (i) Aggregation bias — The mean return averages over many truncated episodes with near-zero reward and a few successful ones; the variance is large ($\sigma \approx 0.63$), so the mean is noisy and not strictly monotone in

success rate. (ii) Penalty accumulation from curiosity-driven actions — The full-curiosity agent explores more aggressively (especially early), which leads to more invalid moves (e.g., wall bumps) and thus more small negative penalties (-0.1 per invalid step) before the policy locks onto the optimal path. Let n_{inv} denote the number of such penalties along a trajectory; even when a run succeeds, the episode return is roughly $1 - 0.1 n_{\text{inv}}$ (ignoring other tiny terms). Therefore the same exploratory behaviour that raises the success rate also drags down the mean extrinsic return. This explains why HMM+full achieves the best success yet a slightly lower episode return than RND or “dyn only” in this simple room.

- **Skill-boundary entropy is inert.** The boundary bonus is gated by $\mathbb{I}\{H_t - H_{t-1} > \varepsilon\}$ and uses filtered entropies $H_t = -\sum_k \alpha_{t,k} \log \alpha_{t,k}$. In Room-15×15 the optimal behaviour is a single locomotion macro-skill (walk straight to the staircase). The forward-backward posteriors quickly concentrate on a dominant state, so H_t remains flat and the gate rarely triggers (Fig. 4.10c ≈ 0).
- **Transition novelty quickly vanishes.** The off-diagonal novelty term rewards unlikely edges under $\hat{A}$ (transition matrix). In this map the optimal path contains very few true skill switches; once those edges are observed a few times, their negative log-potentials drop and the bonus decays to zero (Fig. 4.10d).
- **Dynamics surprise is the only active curiosity signal.** Consequently, “dyn only” and “full curiosity” are nearly identical here. Their improvement over “no intrinsic” comes from better early coverage and faster discovery of the goal rather than from complex skill structuring.

To summarise, the HMM prior already helps by stabilising the representation used by PPO, which is consistent with our motivation for skill preservation. In a single-skill task, boundary and transition bonuses are mostly silent and the dynamics-prediction term is the workhorse. Success rate is the reliable metric in sparse-reward rooms as episode-return means have high variance and are sensitive to episode-length effects and aggregation. We therefore use success rate as the primary criterion in the room and defer to richer intrinsic signals on more structured maps (e.g., River), where multiple skills are required and the HMM-based bonuses become informative.

4.3.2 MiniHack River: transfer from Room and ablation analysis

After training MiniHack Room environments, we used the trained models to train a more difficult game — MiniHack River. The agent still has 8 action keys. But it has to navigate to a boulder , push it into the river  to make it a crossable floor . Sometimes the agent needs to do it multiple times if the river is wide. The goal  is hidden until crossing the river. This introduces at least two macro-skills and their transitions: NAVIGATE $\leftrightarrow$ PUSH. Rewards remain sparse and invalid moves (e.g., bumping into walls or objects) incur small negative penalties. Fig 4.11 shows a demonstration from human players.

We selected 5 models to perform the River game with same seed and 200,000 steps. The results are surprisingly good and summarised in Tab 4.2.

Same as Room environments, we plotted the success rate comparison for MiniHack River environments in Fig 4.12 and the intrinsic rewards for each ablation tests in Fig 4.13.

MiniHack-River-Narrow-v0	Success Rate (%)	Episode Extrinsic Return	Episode Length
No HMM, No intrinsic	38.58	0.323 ± 0.486	242.2 ± 140.6
No HMM,dynamic surprise only	37.22	0.326 ± 0.485	251.3 ± 133.6
HMM, dynamic surprise only	46.93	0.240 ± 0.544	246.9 ± 122.6
HMM, full curiosity	42.69	0.230 ± 0.522	252.1 ± 124.5
HMM, RND	37.90	0.145 ± 0.527	263.6 ± 121.6

Table 4.2: MiniHack River Statistics

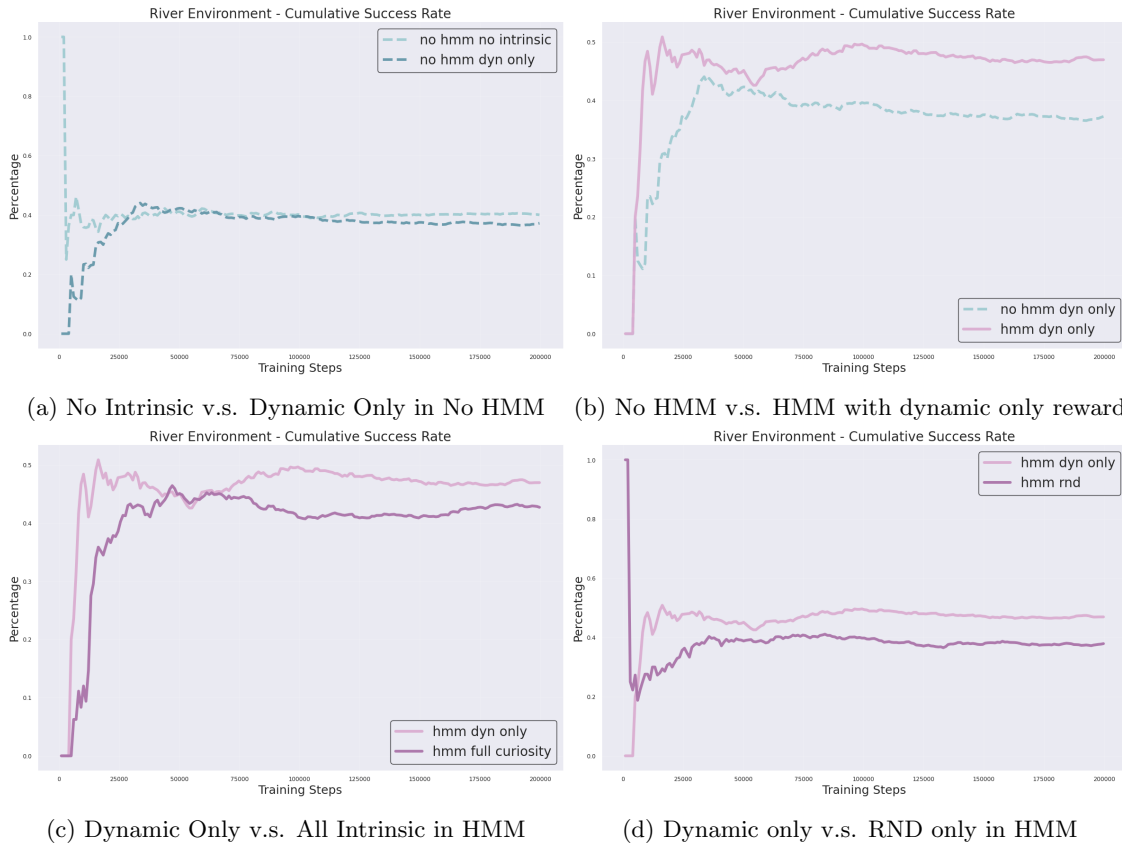


Figure 4.12: Ablation Study of effects of HMM and Intrinsic Reward in MiniHack River.

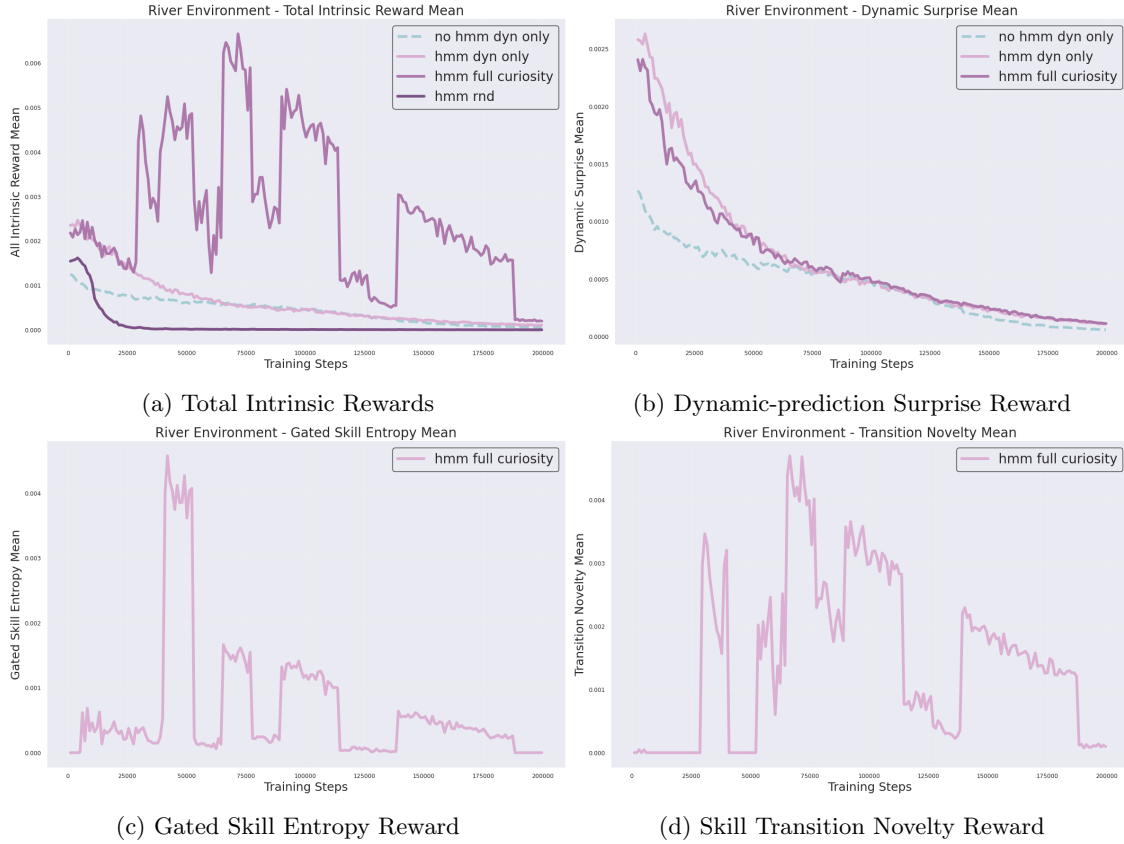


Figure 4.13: Intrinsic Reward Means in MiniHack River.

We have following observation and analysis:

- **HMM + dynamics-only curiosity has the best result (46.93%).** It outperforms both the no-HMM baselines (38.58% and 37.22%) and HMM + full curiosity (42.69%). Episode-length differences are small (~ 5 -20 steps) but HMM+RND produces the longest rollouts (263.6 steps) and the lowest mean return (0.145 ± 0.527).
- **Skill-preserving representation stabilises PPO.** The sticky HDP-HMM keeps the latent z_t piecewise-stationary and supplies filtered beliefs α_t , which reduces input non-stationarity for online PPO. Transferring from Room gives a strong locomotion skill immediately useful in River. This is visible in Fig. 4.12b: the HMM variant climbs faster and plateaus higher than its no-HMM counterpart.
- **Dynamics surprise targets the right novelty.** The dynamics KL is large exactly when the world model is uncertain about interactions (boulder contact, push direction, water-to-floor change), not just unseen coordinates. This drives practice on the push mechanics early (Fig. 4.13b), improving the chance of completing the multi-step plan.
- **Low interference from other bonuses.** With only the dynamics term active, exploration pressure remains goal directed (learn the contact model, then cross), avoiding spurious incentives to oscillate at skill boundaries or along rare transitions.

- The skill-aware bonuses (boundary entropy and transition novelty) become active in River — unlike Room — as the agent alternates between NAVIGATE and PUSH. This is confirmed by the non-zero traces in Fig. 4.13c-d. However, these signals can over-reward boundary behaviour: lingering near the boulder/river edge increases filtered entropy and repeatedly triggers unlikely off-diagonal transitions, which sometimes encourages dithering instead of decisive pushes. The result is a modest drop in success (42.7% v.s. 46.9%) and a lower mean extrinsic return (0.230 v.s. 0.240). Moreover, the full-curiosity policy accrues more invalid-move penalties (e.g., wall/boulder bumps) during such dithering, further depressing return even on successful runs.
- Without the HMM prior, the world-model latent drifts during online training, making the dynamics KL non-stationary and less reliable as an exploration guide. This produces bursts of curiosity around walls/objects that do not translate into the structured push sequence, yielding a slightly lower success (37.2%) than no-intrinsic (38.6%) despite higher early intrinsic reward (Fig. 4.12a, Fig. 4.13b). In short, curiosity without a stable skill-space can push the agent into unproductive novelty.
- RND rewards state novelty regardless of task semantics. In River, the solution requires repeating a specific contact sequence at a fixed location, while RND encourages broad coverage—more wandering and more invalid bumps. Hence the longest episodes (263.6 steps) and the lowest mean return (0.145) in the HMM+RND configuration, with no gain in success (37.9%).
- The best method (HMM+dynamics) has the highest success but not the highest mean extrinsic return. This is not due to longer trajectories (episode lengths are comparable); rather, the full-curiosity and RND agents accumulate more small negative penalties from curiosity-driven bumping, dragging down the mean return. In sparse-reward, contact-heavy tasks, success rate is the more faithful metric, while mean returns are sensitive to penalty counts and carry large variance.

To summarise, the HMM prior is essential for transferring locomotion and stabilising curiosity. The dynamics prediction surprise pinpoints the contact dynamics that matter for solving River, and with HMM it yields the best success. Skill-aware bonuses are active in River (unlike in Room) but require careful weights/annealing to avoid boundary dithering. Novelty-only exploration (RND) is mismatched to the sequential manipulation task demands.

Chapter 5

Discussion

Our results demonstrate that a Bayesian skill space (sticky HDP-HMM) layered on a world model can stabilise on-policy PPO and enable skill-aware curiosity. At the same time, several limitations and opportunities became clear from the analysis in Chapter 4.3.1–4.3.2. We outline concrete, technically grounded directions below.

5.1 Broader task coverage and transfer

- **From navigation to manipulation.** We only trained on navigation-centric maps (Room, River). A natural next step is to target manipulation-heavy levels requiring multi-step preconditions, e.g. *pick up key* $\rightarrow$ *navigate to door* $\rightarrow$ *open door* $\rightarrow$ *exit*. These tasks explicitly exercise skill sequencing, termination, and re-use, which are the strengths of a skill-space model.
- **Multi-task pretraining and transfer.** We can pretrain the world model and skill space on a suite of MiniHack maps (navigation + manipulation), then fine-tune on held-out tasks. This evaluates whether the HMM discovers reusable skills, and whether planning over the skill graph transfers across maps.

5.2 Stronger world models for NetHack

- **Richer encoders.** Replace the shallow GRU with spatially aware: Vision-Language Model, or Vision-Language-Action model [56], and fuse glyphs/colours/messages rather than only the cropped map. Improve temporal credit assignment and long-range memory (doors, keys seen earlier).
- **Object-centric modeling.** Use slot-attention or entity-centric encoders so that the world model reasons over objects (boulders, doors, water) with disentangled states. This should make the skill emissions μ_k interpretable (“PUSH-BOULDER”, “OPEN-DOOR”) and reduce representation drift.

- **Action-conditional and contrastive training.** Train the dynamics to predict z_{t+1} given (z_t, a_t) with both reconstruction and contrastive objectives (CPC/InfoNCE) [15] to stabilise latent geometry while improving one-step prediction used by the dynamics surprise.

5.3 Improving the skill space

- **Explicit durations (HSMM).** Our dwell-time analysis (Chapter 4.2) shows heavy tails for some states; geometric dwell times are sometimes insufficient. Replace the sticky HMM with a hierarchical semi-Markov model (HDP-HSMM) that learns duration distributions (e.g. negative binomial), improving boundary localisation and reducing over-/under-segmentation.
- **Context-conditioned transitions.** Let the transition row $\Phi_{h_{t-1}}$ depend on slow context (e.g. room topology, inventory flags) via a small network. This yields an input-dependent HMM where the skill graph adapts to the local map, while retaining Bayesian posteriors for filtering.
- **Granularity control.** Introduce a penalty or prior on the number of active skills (e.g. a sparsity prior on π) and a merge/split proposal [34] during VI. This “grinds” the skill library so granularity matches semantic macro-actions rather than over-fragmenting locomotion.

5.4 Skill-aware control

- **Options over skills.** Turn each HMM state into an option with a skill-conditioned policy $\pi(a \mid z, h)$ and a learned termination $\beta_h(z)$, with a higher-level policy over skills. This couples discovery (HMM) with execution (options) and should improve credit assignment on multi-step manipulation.
- **Planning in the skill graph.** Using the filtered belief α_t and the estimated transition matrix, perform short-horizon lookahead (beam search/Monte-Carlo Tree Search (MCTS)) in skill space, evaluating rollouts with the value function. This adds model-based guidance while keeping low-level control model-free.
- **Adaptive intrinsic weighting.** Replace fixed η_t schedules by a bandit or learning-progress controller that increases the dynamics bonus when prediction error is large and damps boundary/transition bonuses when they induce dithering (cf. MiniHack River).

5.5 Stability for online learning

- **Online VI with tempered posteriors.** We used a simple ρ -tempering to set the HMM posterior as the next prior. A better approach is to learn the tempering schedule (meta-optimize over ELBO/success AUC) and alternate mini-batch VI with PPO using staggered updates to avoid oscillations.

5.6 Evaluation and analysis at scale

- **Online success and AUC.** We can report on-policy success per update and area under the curve (AUC) as primary metrics, complemented by frozen-policy evaluations with fixed sampling rules.
- **Skill semantics metrics.** Quantify alignment between discovered skills and human-interpretable events (contact, door-open) using mutual information, silhouette scores in latent space, and dwell-fit diagnostics.
- **Robustness and generalisation.** Stress-test across seeds, map generators, and observation perturbations (e.g. hidden goals, clutter). Evaluate transfer from navigation to manipulation to measure compositionality.

5.7 Additional directions

- **Language grounding.** We can incorporate NetHack message text into the world model; align skill boundaries to textual events using weak supervision (contrastive “message $\leftrightarrow$ skill” losses).
- **Open-loop skill curricula.** Automatically generate subgoals by sampling short paths in the skill graph and training the policy to realise them (Go-Explore style), then fine-tune with closed-loop PPO.

The empirical pattern we observed—HMM stabilises online learning; dynamics prediction surprise drives task-relevant exploration; boundary/transition bonuses help when carefully gated—suggests a promising model-based & skill-based RL direction: learn a structured, Bayesian skill library once, then plan and compose those skills with light-weight curiosity on new tasks. The directions above aim to turn this into a scalable recipe for NetHack-style environments and beyond.

Chapter 6

Conclusion

This thesis investigated how to endow a policy with sequential skills that can be preserved and reused while learning online in NetHack/MiniHack. We proposed a hierarchical, Bayesian formulation that couples a VAE world model with a sticky HDP-HMM over latents. The HMM introduces an unbounded library of temporally persistent skills whose Gaussian emissions act as a learned, structured prior for the VAE. We derived an efficient mean-field variational scheme with truncated sticks and a degenerate posterior for the global weights, provided VI-consistent filtering (forward messages) for online use, and defined a suite of skill-aware curiosity bonuses: a dynamics-prediction surprise, a gated skill-boundary entropy, and a transition-novelty term. A three-stage training recipe alternates fitting the VAE and HMM on replay, anneals the HMM prior into the encoder to prevent representation drift, and then trains PPO online with periodic world-model and HMM updates.

On MiniHack Room and MiniHack River we showed that the HMM prior stabilises on-policy PPO and improves success. In Room, the best score was achieved by HMM+full curiosity (highest success and shortest episodes); the extra curiosity does not inflate return because the agent accumulates more small penalties from exploratory wall/boulder bumps—a trade off that is expected under sparse rewards. In River, which requires alternating macro-skills (NAVIGATE/PUSH), HMM+dynamics-only achieved the highest success; adding boundary/transition terms became active but sometimes induced boundary dithering, slightly lowering success and return. Across both maps the dynamics KL proved to be the most robust exploration signal, whereas RND (novelty without structure) encouraged long, unproductive wandering in River.

We developed diagnostics that make the latent structure visible and auditable: PCA/t-SNE plots of skill-coloured latents; MI/TC/DW decompositions with a careful treatment of mixture priors (noting that MI+TC+DW only holds for factorised priors); and HMM fit reports (transition heatmaps, dwell-time goodness-of-fit, occupancy, and Viterbi rasters). These tools confirmed that the HMM prior yields compact, well-separated skill clusters, shrinks variances and reduces axis blow-ups typical of VAEs with an isotropic prior, thereby mitigating non-stationarity seen by PPO.

Our world model is shallow GRU and does not yet fuse all channels object-centrally; sensitivity to curiosity weights can cause over-exploration near boundaries; and experiments were limited to two maps.

A Bayesian skill space from a sticky HDP-HMM, used as a prior over VAE latents and leveraged via simple filtering, is an effective way to stabilise on-policy learning and to focus curiosity on task-relevant novelty. The approach learns a reusable, interpretable skill library and improves success on both navigation and manipulation-flavoured tasks. The proposed diagnostics explain why it works, and the ablations show when each intrinsic signal helps. Building on these results—with richer world models, duration-aware HSMMs, skill-aware control (options/planning over the skill graph), adaptive intrinsic scheduling, and broader multi-task evaluation—offers a promising path to scalable, compositional agents for NetHack-like domains.

Appendix A

NetHack Basic

A.1 blstats Description

#	Field Name	Description
0	x_coordinate	Current x position
1	y_coordinate	Current y position
2	strength_25	Strength (base 25 scale)
3	strength_125	Strength (base 125 scale)
4	dexterity	Dexterity
5	constitution	Constitution
6	intelligence	Intelligence
7	wisdom	Wisdom
8	charisma	Charisma
9	score	Current score
10	hitpoints	Current hit points (capped at 9999)
11	max_hitpoints	Maximum hit points (capped at 9999)
12	depth	Current dungeon depth
13	gold	Amount of gold carried
14	energy	Current magical energy
15	max_energy	Maximum magical energy
16	armor_class	Armor class (lower is better)
17	monster_level	Monster level
18	experience_level	Experience level
19	experience_points	Experience points
20	time	Game time (turn count)
21	hunger_state	Hunger state (0=Satiated, 1=Not Hungry, etc)
22	carrying_capacity	Carrying capacity (% used, 0=Unencumbered, higher=more encumbered)
23	dungeon_number	Which dungeon branch you're in
24	level_number	Level within the current dungeon
25	condition_mask	Bitmask of status conditions. See A.2 for detail
26	Alignment	Character alignment (-1=Chaotic, 0=Neutral, 1=Lawful)

Table A.1: List of the `blstats` vector

Condition mask is coded as bit value. In `blstats`, it is shown as integer in decimal. A hero can have multiple conditions. For example, if the hero is both blind and confused, then `condition_mask = 0x00000020 | 0x00000100 = 0x00000120 = 288`.

Bit Value	Condition	Description
0x00000001L	Stoned	Player is turning to stone (fatal)
0x00000002L	Slimed	Player is turning to slime (fatal)
0x00000004L	Strangled	Player is being strangled (fatal)
0x00000008L	Food Poisoning	Player has food poisoning (fatal)
0x00000010L	Terminal Illness	Player has terminal illness (fatal)
0x00000020L	Blind	Player cannot see
0x00000040L	Deaf	Player cannot hear
0x00000080L	Stunned	Player is stunned
0x00000100L	Confused	Player is confused
0x00000200L	Hallucinating	Player is hallucinating
0x00000400L	Levitating	Player is levitating
0x00000800L	Flying	Player is flying
0x00001000L	Riding	Player is riding a steed

Table A.2: List of the `condition_mask`

A.2 Self-defined Semantic Class

We categorise character $\times$ colour pairs into 17 classes based on their semantics.

Enum Value	Name	Description
0	UNKNOWN	Default/unrecognized elements
1	WALLS	Walls and barriers
2	DOORS	Doors
3	FLOORS	Walkable floor tiles
4	UP STAIRS	Stairs and ladders
5	DOWN STAIRS	Stairs and ladders
6	WATER / LAVA	Water and lava
7	TRAPS	All types of traps
8	ITEMS	Pickup-able items (weapons, armor, etc.)
9	FURNITURE	Non-movable environment objects
10	PLAYER / HUMAN	Player character or human-like entities
11	HARMLESS MONSTERS	Living creatures that do not pose a threat
12	LOW THREAT MONSTERS	Low-threat monsters (e.g. rats, kobolds)
13	MEDIUM THREAT MONSTERS	Medium-threat monsters (e.g. goblins, orcs)
14	HIGH THREAT MONSTERS	High-threat monsters (e.g. dragons, demons)
15	EXTREME THREAT MONSTERS	Extreme-threat monsters (e.g. demons, liches)
16	UNKNOWN MONSTERS	Monsters with unknown threat level

Table A.3: List of Semantic Class

Appendix B

Sticky HDP-HMM Variational Inference

B.1 Row-specific Transition Matrix Φ_k

By Variational Bayes, we have

$$\begin{aligned}\log q^*(\Phi_k) &= \log p(\Phi_k \mid \pi^*) + \mathbb{E}_{q(h_{1:T})} \left[\sum_{t: h_{t-1}=k}^T \log p(h_t \mid h_{t-1}, \Phi_k) \right] + \text{const} \\ &= \sum_{j=1}^{K+1} \left(\alpha \pi_j^* + \kappa \delta_{jk} + \sum_{t=1}^{T-1} \hat{\xi}_{tkj} - 1 \right) \log \Phi_{kj} + \text{const}.\end{aligned}$$

So we have variational update:

$$\hat{\varphi}_{kj} = \alpha \pi_j^* + \kappa \delta_{jk} + \sum_{t=1}^{T-1} \hat{\xi}_{tkj}.$$

B.2 HMM Forward-backward Message Passing

For HMM, we have

$$\begin{aligned}\log q^*(h_{1:T}) &= \mathbb{E}_{q_{-h}} \left[\log p(h_1 \mid \pi^*) + \sum_{t=2}^T \log p(h_t \mid h_{t-1}, \Phi) + \sum_{t=1}^T \log p(z_t \mid h_t, \eta) \right] \\ &= \sum_k \mathbf{1}\{h_1 = k\} \log \pi_k^* + \sum_{t=2}^T \log \hat{A}_{h_{t-1}h_t} + \sum_{t=1}^T \log \hat{B}_{t,h_t} + \text{const},\end{aligned}$$

where $\hat{A}$ is the transition matrix, $\hat{B}$ is emission function such that

$$\log \hat{A}_{ij} = \psi(\hat{\varphi}_{ij}) - \psi \left(\sum_m \hat{\varphi}_{im} \right),$$

and

$$\begin{aligned}
\log \hat{B}_{t,k} &= \mathbb{E}_{q(\eta)q(z)} [\log p(z_t \mid h_t = k, \eta)] \\
&= \frac{1}{2} \mathbb{E}_{q(\eta_k)q(z)} [\log |\Lambda_k| - (z_t - \mu_k)^\top \Lambda_k (z_t - \mu_k)] - \frac{D}{2} \log 2\pi \\
&= \frac{1}{2} \mathbb{E}_{q(\Lambda_k)} [\log |\Lambda_k|] - \frac{1}{2} \mathbb{E}_{q(\eta_k)q(z)} [(z_t - \mu_k)^\top \Lambda_k (z_t - \mu_k)] - \frac{D}{2} \log 2\pi
\end{aligned}$$

where

$$\begin{aligned}
\mathbb{E}_{q(\Lambda_k)} [\log |\Lambda_k|] &= \sum_{d=1}^D \psi \left(\frac{\hat{\nu}_k + 1 - d}{2} \right) + D \log 2 - \log |\hat{\Psi}_k|, \\
\mathbb{E}_{q(\Lambda_k)} [\Lambda_k] &= \hat{\nu}_k \hat{\Psi}_k^{-1},
\end{aligned}$$

and

$$\mathbb{E}_{q(\eta_k)q(z)} [(z_t - \mu_k)^\top \Lambda_k (z_t - \mu_k)] = \text{Tr} \left(\hat{\nu}_k \hat{\Psi}_k^{-1} (\text{diag}(\sigma_t^2) + F_t F_t^\top) \right) + (\mu_t - \hat{\mu}_k)^\top \hat{\nu}_k \hat{\Psi}_k^{-1} (\mu_t - \hat{\mu}_k) + \frac{D}{\hat{\kappa}_k}.$$

The optimal $q(h_{1:T})$ is exactly the posterior distribution of an HMM with initial probability π^* , transition matrix $\hat{A}$ and time-varying emission likelihoods $\hat{B}_{t,k}$. So by (2.20) we have:

- Forward messages: $\alpha_1(k) = \pi_k^* \hat{B}_{1,k}$, $\alpha_{t+1}(k) = \hat{B}_{t+1,k} \sum_{i=1}^{K+1} \hat{A}_{ik} \alpha_t(i)$,
- Backward messages: $\beta_T(k) = 1$, $\beta_t(k) = \sum_{i=1}^{K+1} \hat{A}_{ki} \hat{B}_{t+1,i} \beta_{t+1}(i)$,
- Marginals:
$$\hat{r}_{tk} = q(h_t = k) = \frac{\alpha_t(k) \beta_t(k)}{\sum_i^{K+1} \alpha_t(i) \beta_t(i)},$$

$$\hat{\xi}_{tjk} = q(h_t = j, h_{t+1} = k) = \frac{\alpha_t(j) \hat{A}_{jk} \hat{B}_{t+1,k} \beta_{t+1}(k)}{\sum_{p,q} \alpha_t(p) \hat{A}_{pq} \hat{B}_{t+1,q} \beta_{t+1}(q)}.$$

B.3 Emission Parameters η_k

For $q(\eta_k)$, we have:

$$\begin{aligned}
\log q^*(\eta_k) &= \log p(\eta_k) + \mathbb{E}_{q(h)q(z)} \left[\sum_{t:h_t=k}^T \log p(z_t \mid h_t, \eta) \right] \\
&= \log p(\eta_k) + \sum_{t=1}^T \hat{r}_{tk} \left[\frac{1}{2} \log |\Sigma_k^{-1}| - \frac{1}{2} \text{Tr} (\Sigma_k^{-1} (\Sigma_t + \mu_t \mu_t^\top - 2\mu_t \mu_k + \mu_k \mu_k^\top)) \right].
\end{aligned}$$

We collect

- Expected sample size: $N_k = \sum_{t=1}^T \hat{r}_{tk}$,
- First moment: $S_{1,k} = \sum_{t=1}^T \hat{r}_{tk} \mu_t$,
- Second moment: $S_{2,k} = \sum_{t=1}^T \hat{r}_{tk} (\Sigma_t + \mu_t \mu_t^\top)$.

The parameter updates are

$$\begin{aligned}\hat{\kappa}_k &= \kappa_0 + N_k, \\ \hat{\nu}_k &= \nu_0 + N_k, \\ \hat{\mu}_k &= \frac{\kappa_0 \mu_0 + S_{1,k}}{\hat{\kappa}_k}, \\ \hat{\Psi}_k &= \Psi_0 + S_{2,k} + \kappa_0 \mu_0 \mu_0^\top - \hat{\kappa}_k \hat{\mu}_k \hat{\mu}_k^\top.\end{aligned}$$

B.4 Optimisation on Global Stick-breaking weights π

As discussed before, we can optimise global stick-breaking parameter β and calculate π from β .

Up to constants, the ELBO restricted to β is

$$\begin{aligned}\mathcal{L}(\beta) &= \mathbb{E}_q[\log p(\beta) + \sum_k^{K+1} \log p(\Phi_k | \beta) + \log p(h_1 | \beta)] \\ &= (\gamma - 1) \sum_{k=1}^K \log(1 - \beta_k) + \sum_{k=1}^{K+1} \sum_{j=1}^{K+1} \left[(\alpha \pi_j + \kappa \delta_{jk} - 1) \left(\psi(\hat{\varphi}_{kj}) - \psi \left(\sum_{m=1}^{K+1} \hat{\varphi}_{km} \right) \right) - \log \Gamma(\alpha \pi_j + \kappa \delta_{jk}) \right] \\ &\quad + \sum_{k=1}^{K+1} \hat{r}_{1k} \log \pi_k,\end{aligned}$$

where

$$\pi_k = \beta_k \prod_{l < k} (1 - \beta_l), \quad k \leq K$$

and

$$\pi_{K+1} = 1 - \sum_{k \leq K} \pi_k.$$

Since $\beta_k \sim \text{Beta}(1, \gamma)$, $\beta_k \in (0, 1)$. We can transform β_k to unconstrained variable u_k such that $\beta_k = \sigma(u_k)$ where $\sigma(\cdot)$ is sigmoid function mapping $\mathbb{R}$ to $(0, 1)$.

Appendix C

Training Hyperparameters

C.1 Stage I: VAE Pretraining

- **Dataset:** `nle.dataset` processed replays ($32 \text{ games} \times 32 \text{ sequences per batch}$).
- **Batches:** 100.
- **Batch size:** 32 games per batch.
- **Sequence length:** 32 steps.
- **Epochs:** 15.
- **Warm-up ratio:** 20%
- **Prior:** standard Normal $\mathcal{N}(0, I)$.
- **Masking:** non-map regions of the screen set to zero.
- **Ego window size:** 11.
- **Optimizer parameters:**
 - Optimiser: AdamW
 - Max Learning rate: 1×10^{-3}
 - Weight decay: 10^{-4}
 - Learning rate scheduler: Onecycle
 - Total steps: 100×15
 - `pct_start`: 20%
 - `anneal_strategy`: cosine
 - `div_factor`: 2.0
 - `final_div_factor`: 5.0
 - `cycle_momentum`: False

- **Loss coefficients:**

- $\beta_{MI} = 1.0$
- $\beta_{TC} = 0.0 \rightarrow 10.0$ linear increase in warm-up epochs, constant for the rest
- $\beta_{DW} = 0.2 \rightarrow 1.0$ linear increase in warm-up epochs, constant for the rest
- free bits: 0.75
- encoder dropout: 10%
- decoder dropout: 10%
- loss weights:
 - * ego_class: 5.0,
 - * passibility: 5.0,
 - * safety: 5.0,
 - * reward: 0.5,
 - * done: 1.0,
 - * value_k: 1.0,
 - * bag: 3.0,
 - * stats: 0.5,
 - * msg: 1.0,
 - * goal: 3.0,
 - * occupy: 0.5,
 - * forward: 5.0,
 - * inverse: 10.0,
 - * ego_char: 0.5,
 - * ego_color: 0.5,
 - * hero_loc: 20.0

C.2 Stage II: Alternating HMM + VAE Refinement

- **Dataset:** 32 games $\times$ 3200 steps (all latent $\{z_t\}$).
- **Skills:** 40.
- **Annealing weights:** $\{\lambda_m\}_{m=1}^4$ cosine increased $0.3 \rightarrow 0.4 \rightarrow 0.6 \rightarrow 1.0 \rightarrow 1.0$.
- **MI beta:** linear increased $0.2 \rightarrow 0.333 \rightarrow 0.467 \rightarrow 0.6 \rightarrow 0.6$.
- **TC beta:** linear increased $0.0 \rightarrow 0.067 \rightarrow 0.133 \rightarrow 0.2 \rightarrow 0.2$.
- **DW beta:** linear increased $0.5 \rightarrow 0.667 \rightarrow 0.833 \rightarrow 1.0 \rightarrow 1.0$.
- **Sticky HDP-HMM priors:**
 - Concentration α : 5.0.

- Stickiness κ : 1.0.
- Top-level DP mass γ : 5.0.
- **NIW hyperparameters:** ($\mu_0 = 0, \kappa_0 = 1, \Psi_0 = 30, \nu_0 = 106$).
- **π iterations:** 200.
- **π learning rate:** 1×10^{-3} .
- **VAE retraining:** same as pre-training unless stated above.

C.3 Stage III: Online PPO with VAE + HMM

- **Policy:** PPO with GRU hidden size 256.
- **Number of parallel environments:** 8.
- **Rollout length:** 128 steps.
- **PPO hyperparameters:**
 - Learning rate: 3×10^{-4} .
 - Minibatch env: 4.
 - Epochs per update: 4.
 - Discount γ : 0.99.
 - GAE λ : 0.95.
 - Clip coefficient: 0.2.
 - Value loss coefficient: 0.5.
 - Entropy coefficient: 0.01.
 - Max gradient norm: 0.5.
- **Intrinsic reward configuration:**
 - Dynamics KL: $\eta_0 = 0.03$, $\tau = \text{total steps} / \log(20)$, squash $\tau = 180.0$ if enable hmm else 50.0.
 - Skill entropy: $\eta_0 = 0.002$, $\tau = \text{total steps} / \log(20)$, squash $\tau = 1.0$.
 - Skill-transition novelty: $\eta_0 = 0.0005$, $\tau = \text{total steps} / \log(20)$, squash $\tau = 1.0$.
 - RND: disabled by default; if enabled, learning rate 1×10^{-3} , $\eta_0 = 0.002$, $\tau = \text{total steps} / \log(20)$, squash $\tau = 1.0$.
 - EMA β : 0.99, ε : 1.0×10^{-8} .
- **HMM online update:**
 - Every 5,120 environment steps, with growth 1.2.
 - Update every cap: 12,000.

- Fit window: 400,000.
- Max batch size: 128.
- Max iteration: 5.
- ρ_{emission} : 0.05.
- $\rho_{\text{transition}}$: 0.05.
- π steps: 10.
- π learning rate: 5.0×10^{-4} .
- Temper weight: 0.2

- **VAE online update:**

- same as HMM
- Span length: 64.
- 32 spans $\times$ 4 steps per refresh
- Learning rate: 3.0×10^{-5} .
- β : 0.1 for all

Appendix D

Declaration on LLM Use

In this work, LLM was used to do literature review, summarisation, proofread and polish of my raw work and sanity check my maths derivation. LLM was also used for plotting results, logging and debugging code.

Bibliography

- [1] Y. Li, “Deep reinforcement learning: An overview,” *arXiv preprint arXiv:1701.07274*, 2017.
- [2] Y. Guo, J. Choi, M. Moczulski, S. Feng, S. Bengio, M. Norouzi, and H. Lee, “Memory based trajectory-conditioned policies for learning from sparse rewards,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 4333–4345, 2020.
- [3] G. Wang, M. Xin, W. Wu, Z. Liu, and H. Wang, “Learning of long-horizon sparse-reward robotic manipulator tasks with base controllers,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 3, pp. 4072–4081, 2022.
- [4] T. Salimans and R. Chen, “Learning montezuma’s revenge from a single demonstration,” *arXiv preprint arXiv:1812.03381*, 2018.
- [5] H. Küttler, N. Nardelli, A. H. Miller, R. Raileanu, M. Selvatici, E. Grefenstette, and T. Rocktäschel, “The nethack learning environment,” 2020.
- [6] M. Samvelyan, R. Kirk, V. Kurin, J. Parker-Holder, M. Jiang, E. Hambro, F. Petroni, H. Küttler, E. Grefenstette, and T. Rocktäschel, “Minihack the planet: A sandbox for open-ended reinforcement learning research,” 2021.
- [7] A. G. Barto and S. Mahadevan, “Recent advances in hierarchical reinforcement learning,” *Discrete event dynamic systems*, vol. 13, no. 4, pp. 341–379, 2003.
- [8] P.-L. Bacon, J. Harb, and D. Precup, “The option-critic architecture,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 31, 2017.
- [9] A. Vezhnevets, S. Osindero, T. Schaul, N. Heess, D. Silver, K. Kavukcuoglu, *et al.*, “Feudal networks for hierarchical reinforcement learning,” in *International Conference on Machine Learning*, 2017.
- [10] K. Gregor, D. Rezende, and D. Wierstra, “Variational intrinsic control,” *arXiv preprint arXiv:1611.07507*, 2017.
- [11] B. Eysenbach, A. Gupta, J. Ibarz, and S. Levine, “Diversity is all you need: Learning skills without a reward function,” in *International Conference on Learning Representations*, 2019.
- [12] A. Barreto, D. Borsa, J. Quan, T. Schaul, D. Silver, M. Hessel, D. Mankowitz, A. Zidek, and R. Munos, “Transfer in deep reinforcement learning using successor features and generalised policy improvement,” in *International Conference on Machine Learning*, pp. 501–510, PMLR, 2018.

- [13] O. E. L. Team, A. Stooke, A. Mahajan, C. Barros, C. Deck, J. Bauer, J. Sygnowski, M. Trebacz, M. Jaderberg, M. Mathieu, *et al.*, “Open-ended learning leads to generally capable agents,” *arXiv preprint arXiv:2107.12808*, 2021.
- [14] D. P. Kingma, M. Welling, *et al.*, “An introduction to variational autoencoders,” *Foundations and Trends® in Machine Learning*, vol. 12, no. 4, pp. 307–392, 2019.
- [15] A. van den Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” 2019.
- [16] D. Ha and J. Schmidhuber, “World models,” *arXiv preprint arXiv:1803.10122*, vol. 2, no. 3, 2018.
- [17] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell, “Progressive neural networks,” *arXiv preprint arXiv:1606.04671*, 2016.
- [18] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the national academy of sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [19] E. B. Fox, E. B. Sudderth, M. I. Jordan, and A. S. Willsky, “The sticky hdp-hmm: Bayesian nonparametric hidden markov models with persistent states,” *Arxiv preprint*, vol. 2, 2007.
- [20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms.” *arXiv preprint arXiv:1707.06347*, 2017.
- [21] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, “Curiosity-driven exploration by self-supervised prediction,” in *International conference on machine learning*, pp. 2778–2787, PMLR, 2017.
- [22] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, “Exploration by random network distillation,” 2018.
- [23] M. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos, “Unifying count-based exploration and intrinsic motivation,” *Advances in neural information processing systems*, vol. 29, 2016.
- [24] C. K. Williams and C. E. Rasmussen, *Gaussian processes for machine learning*, vol. 2. MIT press Cambridge, MA, 2006.
- [25] T. S. Ferguson, “A bayesian analysis of some nonparametric problems,” *The annals of statistics*, pp. 209–230, 1973.
- [26] Y. W. Teh *et al.*, “Dirichlet process,” *Encyclopedia of machine learning*, vol. 1063, pp. 280–287, 2010.
- [27] D. Blackwell and J. B. MacQueen, “Ferguson distributions via pólya urn schemes,” *The annals of statistics*, vol. 1, no. 2, pp. 353–355, 1973.

- [28] J. Sethuraman, “A constructive definition of dirichlet priors,” *Statistica sinica*, pp. 639–650, 1994.
- [29] J. Pitman, *Combinatorial stochastic processes: Ecole d’été de probabilités de saint-flour xxxii-2002*. Springer Science & Business Media, 2006.
- [30] N. Houlsby, F. Huszár, Z. Ghahramani, and M. Lengyel, “Bayesian active learning for classification and preference learning,” 2011.
- [31] S. Gershman, M. Hoffman, and D. Blei, “Nonparametric variational inference,” 2012.
- [32] M. C. Hughes and E. Sudderth, “Memoized online variational inference for dirichlet process mixture models,” *Advances in neural information processing systems*, vol. 26, 2013.
- [33] C. Wang and D. Blei, “Truncation-free online variational inference for bayesian nonparametric models,” *Advances in neural information processing systems*, vol. 25, 2012.
- [34] N. Ueda and R. Nakano, “Split and merge em algorithm for improving gaussian mixture density estimates,” *Neural Networks*, 2000.
- [35] M. Bryant and E. B. Sudderth, “Truly nonparametric online variational inference for hierarchical dirichlet processes,” in *Advances in Neural Information Processing Systems*, 2012.
- [36] M. C. Hughes, W. Stephenson, and E. B. Sudderth, “Scalable adaptation of state complexity for nonparametric hidden markov models,” in *Advances in Neural Information Processing Systems*, 2015.
- [37] M. C. Hughes, “*Reliable and scalable variational inference for nonparametric mixtures, topics, and sequences*”. PhD thesis, Brown University, USA, 2016.
- [38] R. M. Neal and G. E. Hinton, “A view of the em algorithm that justifies incremental, sparse, and other variants,” in *Learning in graphical models*, pp. 355–368, Springer, 1998.
- [39] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” in *Advances in Neural Information Processing Systems*, vol. 12, pp. 1057–1063, 2000.
- [40] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control with generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2016. ICLR 2016.
- [41] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 1861–1870, 2018.
- [42] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” *arXiv preprint arXiv:1509.02971*, 2016. ICLR 2016 (accepted paper).
- [43] F. Chen, G. Datta, S. Kundu, and P. Beerel, “Self-attentive pooling for efficient deep learning,” 2022.

- [44] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” 2016.
- [45] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “Film: Visual reasoning with a general conditioning layer,” 2017.
- [46] R. T. Q. Chen, X. Li, R. Grosse, and D. Duvenaud, “Isolating sources of disentanglement in variational autoencoders,” 2019.
- [47] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling, “Improving variational inference with inverse autoregressive flow,” 2017.
- [48] M. Bryant and E. Sudderth, “Truly nonparametric online variational inference for hierarchical dirichlet processes,” *Advances in Neural Information Processing Systems*, vol. 25, 2012.
- [49] P. Liang, S. Petrov, M. I. Jordan, and D. Klein, “The infinite pcfg using hierarchical dirichlet processes,” in *Proceedings of the 2007 joint conference on empirical methods in natural language processing and computational natural language learning (EMNLP-CoNLL)*, pp. 688–697, 2007.
- [50] L. Itti and P. Baldi, “Bayesian surprise attracts human attention,” *Vision Research*, vol. 49, no. 10, pp. 1295–1306, 2009. Visual Attention: Psychophysics, electrophysiology and neuroimaging.
- [51] E. Hambro, R. Raileanu, D. Rothermel, V. Mella, T. Rocktäschel, H. Küttler, and N. Murray, “Dungeons and data: A large-scale nethack dataset,” 2023.
- [52] Y. Burda, H. Edwards, D. Pathak, A. Storkey, T. Darrell, and A. A. Efros, “Large-scale study of curiosity-driven learning,” in *ICLR*, 2019.
- [53] S. Lobel, A. Tamar, and M. Tennenholtz, “Optimistic initialization for exploration in continuous control,” in *AAAI*, 2022.
- [54] M. Yuan, B. Li, X. Jin, and W. Zeng, “Automatic intrinsic reward shaping for exploration in deep reinforcement learning,” in *ICML*, 2023.
- [55] R. a. Zhang, “Rank the episodes: A simple approach for exploration in procedurally-generated environments,” *arXiv:2101.08152*, 2021.
- [56] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” 2023.